

Copyright
by
Gus Santaella
2026

The Thesis Committee for Gus Santaella
certifies that this is the approved version of the following thesis:

**Learning Robust Strategies for an Orbital Target Defense
Game: Phased Adversarial Reinforcement Learning under
Estimation Uncertainty and Control Constraints**

SUPERVISING COMMITTEE:

Maruthi Akella, Supervisor

David Fridovich-Keil, Reader

**Learning Robust Strategies for an Orbital Target Defense
Game: Phased Adversarial Reinforcement Learning under
Estimation Uncertainty and Control Constraints**

**by
Gus Santaella**

Thesis

Presented to the Faculty of the Graduate School of
The University of Texas at Austin
in Partial Fulfillment
of the Requirements
for the Degree of

Master of Science in Engineering

**The University of Texas at Austin
August 2026**

Dedication

To all future Longhorn graduate students.

Epigraph

What starts here changes the world.

—University of Texas at Austin

Acknowledgments

Thanks to the previous authors of this template, and to the Graduate School for providing it.

Preface

Readers, this document represents a great deal of time and effort. Enjoy.
Cheers, Gus Santaella.

Abstract

Learning Robust Strategies for an Orbital Target Defense Game: Phased Adversarial Reinforcement Learning under Estimation Uncertainty and Control Constraints

Gus Santaella, MSE
The University of Texas at Austin, 2026

SUPERVISOR: Maruthi Akella

Your abstract goes here. Do not use `\abstract!`

Table of Contents

List of Tables	12
List of Figures	13
Chapter 1: Introduction	14
Chapter 2: Literature review	17
Chapter 3: Environment and Spacecraft Dynamics	19
3.1 RL+GNC architecture diagram	20
3.2 Methodology	21
3.2.1 Implementation jargon	21
3.3 Methodology	21
3.3.1 Symbols and Notation	21
3.3.2 Conventions	23
3.3.3 Implementation terminology	23
3.4 Dynamics models	24
3.4.1 HCW (Hill–Clohessy–Wiltshire) orbital relative dynamics - Clohessy and Wiltshire (1960) and Hill (1878)	24
3.4.2 Elliptical LTV orbital relative dynamics - Tschauner and Hempel (1965)	25
3.5 Estimation model - Bearing-Only Extended Kalman Filter (EKF)	29
3.6 Controller model - CBF-QP velocity saturation controller	31
3.7 State, action, and observation.	33
3.8 Reward shaping and termination conditions	35
3.8.1 On behavior cloning for partial observability	36
3.8.2 Partial observability via teacher-student distillation	37
Chapter 4: Learning Methods	38
4.1 Single Phase PPO Algorithm	38
4.1.1 Policy development	40
4.1.2 Advantage Estimation and PPO Objective	41
4.2 Phased Stationary Training Schedule	41
4.2.1 Rule-Based Attacker (Only used for phase 0 defender warm start)	43
4.3 Inference.	44
4.4 Metrics and Evaluation	46
4.5 Metrics and Evaluation	46

Chapter 5: Training Experiments	48
5.1 Trainings Presented on this Thesis	48
5.2 Training case 1: $u_{\max} = 0.1, v_{\max} = 1, v_{\max}^{\text{IC}} = 0.05$	48
5.3 Training case: $u_{\max} = 0.5, v_{\max} = 1.5, v_{\max}^{\text{IC}} = 1.0$	49
5.4 Training case: $u_{\max} = 2.0, v_{\max} = 1, v_{\max}^{\text{IC}} = 1.0$	49
5.5 Sample Rollout Explained in Images	50
Chapter 6: Policy Monte Carlo Testing	51
6.1 Training case: $u_{\max} = 0.1, v_{\max} = 1, v_{\max}^{\text{IC}} = 0.05$	52
6.1.1 HCW	52
6.1.2 Elliptic LTV	54
6.2 Training case: $u_{\max} = 0.5, v_{\max} = 1.5, v_{\max}^{\text{IC}} = 1.0$	56
6.2.1 HCW	56
6.2.2 Elliptic LTV	58
6.3 Training case: $u_{\max} = 2.0, v_{\max} = 1, v_{\max}^{\text{IC}} = 1.0$	60
6.3.1 HCW	60
6.3.2 Elliptic LTV	62
6.4 Analysis of results anecdotes of development	65
6.4.1 The well-defined, zero-sum objective definition via the reward function and the episodic termination conditions properly guide agents towards learning coherent policies	65
6.4.2 The constrained volume in space in which the agent is allowed to learn its policies allows for excellent learning transitions	66
6.4.3 The trained policies to properly interpolate to larger arenas during evaluation, even if it was never trained on observations corresponding to larger arenas.	66
6.4.4 The velocity saturation controller properly bounds the agent's behavior within a "safe" threshold without preventing learning	67
6.4.5 The policies trained in LTI dynamics translate to LTV dynamics with the provided velocity and acceleration bounds	68
6.4.6 Trained policies hold under partial observability and sensor noise when trained via behavior cloning from the full states	68
6.5 Analysis of ideas for future development	69
Chapter 7: Conclusion	71
Appendix A: Monte Carlo Dispersions	73
A.1 Appendix section	73
A.1.1 Appendix section	73
Appendix B: Another Appendix	74

Glossary	75
Works Cited	76
Vita	78

List of Tables

6.1	Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	52
6.2	Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	53
6.3	Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	54
6.4	Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	55
6.5	Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	56
6.6	Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	57
6.7	Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	58
6.8	Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	59
6.9	Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	60
6.10	Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	61
6.11	Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	62
6.12	Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].	63

List of Figures

3.1	Sample of a policy rollout showing the agents carrying the proposed zero-sum game setup (left), as well as a simple graphic describing training scenarios for the defender (right).	20
3.2	Diagram showing the RL + GNC algorithm	20
4.1	Diagram showing the Inference	46
5.1	Combined training return curves for $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$	49
5.2	Combined training return curves for $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$	49
5.3	Combined training return curves for $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$	50

Chapter 1: Introduction

Adversarial scenarios with non-cooperative agents have become a relevant area of research in real-world autonomous systems and robotics problems. Traditional optimal control frameworks to solve these kinds of problems tend to assume certain information strategies which can become jeopardized when an agent is following a shifting, actively adversarial strategy coming from a versatile, intelligent opponent.

In these cases, classical control architectures may prove to be brittle against these highly dynamic strategies. Inspired by this dilemma, we chose to develop a reinforcement-learning (RL) based approach. The benefit behind these RL-based architectures is that, when properly trained, these can produce a single, deterministic policy that can actively react and adapt to a wide arrange of scenarios it's been trained against.

One of the arguments behind why research and industry have not widely adopted RL algorithms within their autonomy/GNC work comes down to their lack of convergence guarantees. This is due to the fact that these algorithms try to optimize against a highly non-convex objective using gradients from noisy trajectories, approximated value functions learned during trainings, their proneness to overfit to the data distribution that they're trained for, and their struggle to interpolate to data outside this distribution. In other words, all of this can lead to a noisy stochastic gradient ascent/descent process that finds a locally optimal solutions, not necessarily the global optimal solution, which can then not work in regimes outside the training dataset. The development of better RL algorithms that can solve these mishaps of the current state-of-the-art is perhaps some of the most interesting research going on in the world of theoretical deep reinforcement learning in this day and age.

Regardless, algorithms such as Proximal Policy Optimization (PPO, for short), an on-policy algorithm created Schulman et al. (2017) at OpenAI, remains the most

widely popular algorithm in RL research for engineering applications. PPO is based on the principles of stochastic gradient ascent on a clipped surrogate objective function. Furthermore, PPO-based architectures are relatively simple to set up, reuse each rollout batch on multiple gradient steps, are easy to parallelize, and, most importantly, provide much more stability than vanilla policy gradient methods. For a controls based application, which is what we’re most interested on in this work, perhaps the biggest benefit of PPO is that it can be used for control continuous and discrete action spaces.

With all of the above in mind, we were interested in developing a staged reinforcement learning (RL) framework to train, model, and rollout robust adversarial policies for orbital zero sum games in Low Earth Orbit (LEO) in closed-loop aerospace guidance, navigation, and control architectures. For our own applications, we wanted to solve a few questions:

- Can an RL architecture use the PPO algorithm learn coherent policies for continuous control of orbital spacecraft in LEO in zero-sum adversarial scenarios?
- Can an RL architecture learn coherent policies under while interacting subject to the constraints of a classical controller in the loop?
- Can an RL architecture learn coherent policies if, instead of having access of the opponent’s full state, it had partial observability and had to rely on measurements in-the-loop?
- Can an RL architecture interpolate outside of the training dataset even if it was never trained against some of that data?
- Can an RL architecture do all the above at once, provide convergent and high-performing policies despite the stochasticity?

With this in mind, we propose the (NAME PENDING) method for training and rollout of RL-enabled aerospace GNC. The method uses dense step per geometry-based

rewards and phased opponent freezing to mitigate non-stationarity encountered in multi-agent training. The method learns while subject to the constraints of a classical velocity-saturating CBF-QP controller and, optionally, partial observability of the opponent’s state via a bearings-only Extended Kalman Filter via behavior cloning.

Our Monte Carlo evaluation results using the resulting policies show that (NAME PENDING) produces highly performing policies in a constrained, zero-sum orbital asset defense game with equal spacecraft characteristics. Furthermore, this is done while providing fast-rollout and compute inference times of (DATA PENDING) per timestep, which is (DATA PENDING) faster than the current state of the art.

Our findings open the way for the development of highly compute efficient RL-based method for autonomous systems research and application, and a discussion of potential directions for future work is provided.

Chapter 2: Literature review

It is no question that these sorts of orbital games have become an ever more prevalent area of research in aerospace autonomy academia, with plenty of examples of related works in recent years mostly on the optimal control-based methods.

- In “*Pursuit–Evasion Game for Satellites Based on Continuous Thrust Reachable Domain*”, Gong et al. (2020), the authors developed a classical differential game of satellite pursuit-evasion under continuous thrust using a Clohessy–Wiltshire (CW) model.
- In “*A Pursuit-Evasion Game in the Orbital Plane*”, Selvakumar and Bakolas (2017), they develop a two player zero-sum differential game with input constraints, providing a semi-analytical capture-time solution to their problem using closed-form optimal controls.
- In “*Multiple-to-One Orbital Pursuit: A Computational Game Strategy*”, Liu et al. (2025), they propose a one-step look-ahead Nash equilibrium method for cooperative pursuit-evasion in cooperative spacecraft.

In addition to more classical approaches to these rigorous space-bound problems, there’s been a rise of journal published research trying to solve variations of these games but opting for online reinforcement learning (RL) approaches in recent years. Despite the long standing criticism for core reinforcement learning algorithms to not be able to provide theoretical convergence guarantees to learned policies, researchers have found problems in which they’ve come in useful and provided empirically convergent policies, strong performance relative to their assigned tasks, and unmatched computational efficiency.

- In *"Online Solution for Orbital Pursuit-Evasion Game via Heterogeneous Proximal Policy Optimization"*, Li and Wang (2025), they successfully developed a PPO framework for multi-agent orbital pursuit-evasion, outperforming shooting methods in their optimality index metrics in computational efficiency while providing robustness to suboptimal opponent strategies.
- In *A Non-Fixed-Duration Turn-Based Orbital Game Algorithm Based on History-Aware and Predictive Result Reward Proximal Policy Optimization*, Cao et al. (2026), they proposed HPRR-PPO algorithm to do alternating learning for non-fixed-duration turn-based orbital game problems, showing effective interception against evaders with compute speed suitable for real time deployment claiming a compute runtime of 1 ms per made decision.
- In *"Near-Optimal Interception Strategy for Orbital Pursuit-Evasion using Deep Reinforcement Learning"*, Zhang et al. (2022), they use deep reinforcement learning to achieve near-optimal guidance laws in one to one orbital pursuit-evasion games.

The fact that all these papers above have been published in the past few years makes it clear that there is growing interest in developing machine learning methods in adversarial orbital dynamics, especially with the growing interest within the autonomy community to implement machine learning-based frameworks to complement or even replace classical control architectures. This suggests the relevancy that this line of work has in the general literature.

Furthermore, it justifies the novelty of this work, given the fact here we're using a combination of different reinforcement learning techniques that can lead to coherent, robust learning strategies in zero-sum orbital defense game settings: using PPO to find optimal strategies per the given set of hyperparameters complex reward shaping, doing so without help of an equilibrium/Nash prior solution, and testing how to implement this method under a partial observability setting via bearings-only Kalman Filter measurements.

Chapter 3: Environment and Spacecraft Dynamics

To test our hypothesis, we propose a constrained zero-sum game, initially based on linear-time-invariant 2nd-order HCW dynamics to model it like an interaction of two agents relative to a chief circular orbit in the Local-Vertical Local-Horizontal (LVLH) frame. We bound these agents into the boundary of an imaginary spherical arena centered at the chief. In our initial setup, we have two agents and a static element in the environment:

1. **Asset of Interest:** A static object in the LVLH frame that is the "chief" of the orbit.
2. **Player 1 (defender):** Defends an asset of interest by preventing player 2 from intercepting the asset of interest by capturing. If it intercepting player 2 or forces it out of the arena, it wins.
3. **Player 2 (attacker):** Attacks the central chief asset of interest by preventing player 1's defense strategy. If it intercepts the asset of interest, forces the defender to crash into it, or forces the defender out of the arena, it wins.

The result is an interesting variation of pursuit evasion scenario where the conditions above make it a zero sum, spatially constrained game that can make it easy for a online RL method to learn.

The states are the positions and velocities of each agent and the actions are continuous control inputs in 3D space, where we assume both spacecraft to be fully actuated in three translational degrees of freedom. For the sake of simplicity, we ignore the attitude evolution in this work.

The entire simulation environment was developed by the author in Python, using the PyTorch library to help with the RL training.

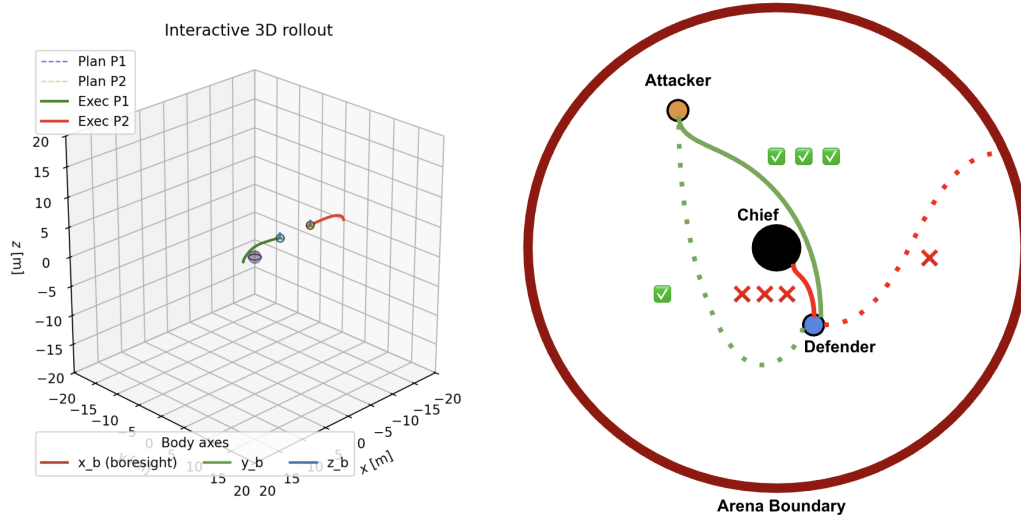


Figure 3.1: Sample of a policy rollout showing the agents carrying the proposed zero-sum game setup (left), as well as a simple graphic describing training scenarios for the defender (right).

3.1 RL+GNC architecture diagram

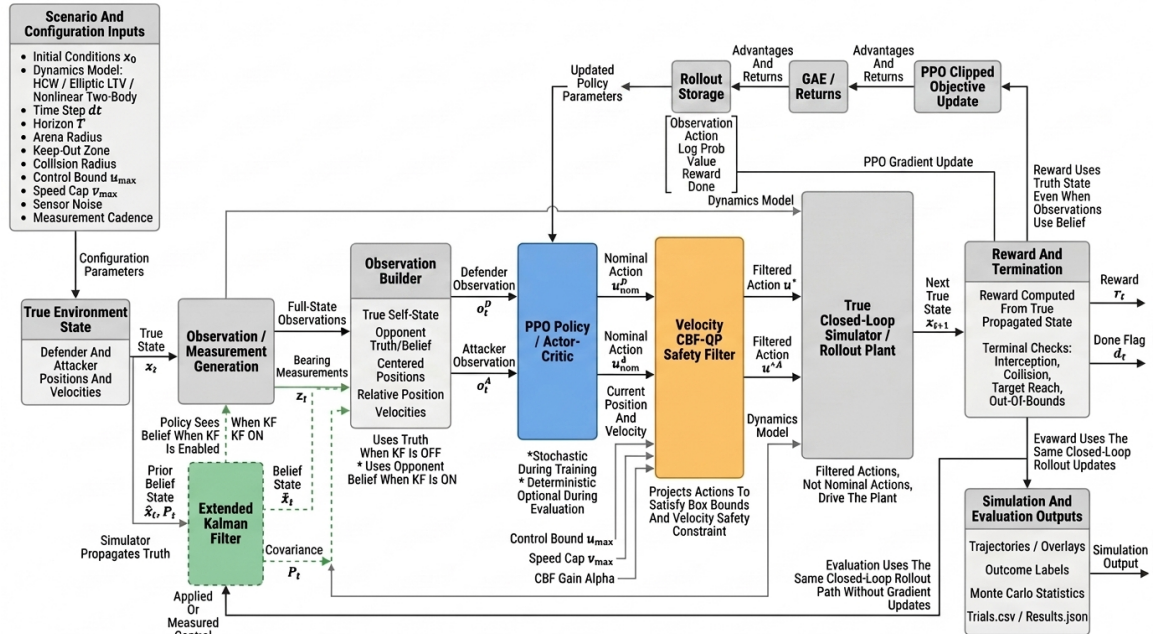


Figure 3.2: Diagram showing the RL + GNC algorithm

3.2 Methodology

3.2.1 Implementation jargon

- **Actor:** residual MLP on ϕ with prior blend $\beta_{\text{def}} \in [0, 1]$, diagonal standard deviation $\sigma = \exp(\log \sigma)$ (this is learned).
- **Squash:** $a = u_{\text{max}} \tanh(u_{\text{raw}})$ with exact log-Jacobian correction.
- **VecEnv:** single-process vectorization, early-terminated learning environments are reset to keep tensor shapes fixed.
- **Regularization:** per-step effort penalties λ_D, λ_A ; soft-wall with threshold ρ_0 and gain κ .
- **Annealing:** $k_{\text{cent}} \leftarrow \gamma_k k_{\text{cent}}$, γ_k linearly decreasing to γ_k^{min} .

3.3 Methodology

3.3.1 Symbols and Notation

$i \in \{1, 2\}$	Agent index (1: defender, 2: attacker).
$D \in \{2, 3\}$	Spatial dimension.
Δt	Discrete simulation timestep.
$R > 0, c \in \mathbb{R}^D$	Arena radius and center.
$p_i, v_i \in \mathbb{R}^D$	Position and velocity of agent i .
$x_i = \begin{bmatrix} p_i \\ v_i \end{bmatrix} \in \mathbb{R}^{2D}$	Agent state; under HCW dynamics, $x_i^+ = A_d x_i + B_d u_i$.
A_d, B_d	Discrete-time HCW state and input matrices.
$u_i \in \mathbb{R}^D$	Commanded acceleration, with $\ u_i\ _{\infty} \leq u_{\text{max}}$.
$s = [x_1^{\top}, x_2^{\top}]^{\top} \in \mathbb{R}^{4D}$	Full environment state.
$o_{\text{full}} \in \mathbb{R}^{5D}$	Full-state observation $[(p_1 - c), (p_2 - c), (p_2 - p_1), v_1, v_2]$.
$\hat{x}_{j i}, \Sigma_{j i}$	Agent i 's EKF estimate of agent j 's state mean and covariance.

$o_{\text{EKF}}^{(i)} \in \mathbb{R}^{5D}$	Agent- i observation formed by replacing the opponent state in o_{full} with EKF estimates.
$Q_{\text{EKF}}, R_{\text{EKF}}$	EKF process and measurement noise covariance matrices.
$d_1 = \frac{\ p_1 - c\ ^2}{R^2}$	Normalized defender-to-asset distance.
$d_2 = \frac{\ p_2 - c\ ^2}{R^2}$	Normalized attacker-to-asset distance.
$r_{\text{rel}}^2 = \frac{\ p_2 - p_1\ ^2}{R^2}$	Normalized squared defender-attacker separation.
$r_{\text{cap}} > 0$	Capture/intercept radius.
$d_{\text{dock}} = \frac{\max\{0, \ p_2 - p_1\ - r_{\text{cap}}\}}{R}$	Normalized docking/separation term used in reward shaping.
$\beta_{\text{cap}}, \beta_{\text{wall}}$	Terminal reward magnitudes for capture and boundary events.
$k_{\text{pos}}, k_{\text{dock}}$	Step-reward weights.
v_{max}	Velocity bound enforced by the safety filter.
α_{CBF}	Control barrier function gain.
$u_{\text{nom}}, u_{\text{cmd}}, u_{\text{app}}$	Nominal, squashed, and applied control inputs.
$P_{\text{pos}} = [I \ 0] \in \mathbb{R}^{D \times 2D}$	Position projector; $F = P_{\text{pos}} B_d$.
$K = (F^\top F + \lambda_{\text{ridge}} I)^{-1} F^\top$	Ridge gain used in the one-step center-pull prior.
$u_{\text{center}} = -K(P_{\text{pos}} A_d x_2 - c)$	One-step center-pull term in the rule-based attacker.
$u_{\text{repulse}} = \frac{g(p_2 - p_1)}{(\max\{r_{\text{min}}^2, \ p_2 - p_1\ ^2\} + \varepsilon_{\text{rep}})^{3/2}}$	Smooth repulsion term in the rule-based attacker.
w_c, w_r, w_d	Weights in the rule-based attacker.
$\gamma \in (0, 1]$	Discount factor.
$\lambda_{\text{GAE}} \in [0, 1]$	Generalized advantage estimation trace parameter.
ϵ_{PPO}	PPO clipping half-width.
$c_V, c_{\mathcal{H}}$	Value-loss and entropy regularization coefficients.
$\mu_\theta(\cdot), \sigma_\theta(\cdot)$	Mean and diagonal standard deviation of the Gaussian policy.

N_{env}, H, B	Number of environments, rollout horizon, and batch size, with $B = N_{\text{env}}H$.
U, E	PPO updates per policy and epochs per update.
θ, ψ	Policy and value-function parameters.

3.3.2 Conventions

$(\cdot)^+$	Quantity at the next discrete time step.
$\hat{(\cdot)}$	Estimated quantity produced by the EKF.
$(\cdot)_{j i}$	State or quantity of agent j as estimated by agent i .
$\text{sat}_{u_{\max}}(\cdot)$	Componentwise saturation onto $[-u_{\max}, u_{\max}]$.
$\text{clip}_{[a,b]}(\cdot)$	Componentwise clipping onto the interval $[a, b]$.
$\ \cdot\ _2, \ \cdot\ _\infty$	Euclidean and max norms, respectively.

3.3.3 Implementation terminology

- **Policy network (actor):** residual MLP that outputs the mean of a diagonal Gaussian policy; the log-standard-deviation vector is learned jointly with the policy.
- **Squashed Gaussian policy:** the raw action u_{raw} is mapped to $u_{\text{cmd}} = u_{\max} \tanh(u_{\text{raw}})$, with the exact log-Jacobian correction used in PPO.
- **Vectorized environment (VecEnv):** N_{env} rollouts are advanced in parallel; terminated environments are reset immediately to preserve fixed tensor shapes within a batch.
- **Fixed-opponent phase:** a training phase in which the learning agent trains against a frozen opponent policy, or a mixture of frozen opponent variants, to reduce non-stationarity.
- **Estimator-enabled rollout:** a rollout setting in which the policy receives EKF-derived opponent estimates while rewards and terminal events are still

computed from the true propagated state.

- **Regularization and annealing:** per-step effort penalties λ_D, λ_A , soft-wall parameters (ρ_0, κ) , and center-pull annealing $k_{\text{cent}} \leftarrow \gamma_k k_{\text{cent}}$ with γ_k scheduled down to $\gamma_k^{\min}$.
- **Teacher and student policies:** the teacher is the full-state policy; the student is the estimator-backed policy trained to imitate the teacher under partial observability.

3.4 Dynamics models

3.4.1 HCW (Hill–Clohessy–Wiltshire) orbital relative dynamics - Clohessy and Wiltshire (1960) and Hill (1878)

We model the follower in the chief agent’s Local Vertical, Local Horizontal (LVLH) frame assuming a circular reference orbit of radius r_0 around a body with parameter μ . The mean motion is defined as:

$$n = \sqrt{\mu/r_0^3}$$

Let (x, y, z) be defined as the relative positions of the agents and let the control thrust acceleration of the agents be defined as: $a = (a_x, a_y, a_z)^\top$. For small separations between agents and assuming the chief follows a circular orbit, the linearized HCW equations are

$$\ddot{x} = 2n\dot{y} + 3n^2x + a_x,$$

$$\ddot{y} = -2n\dot{x} + a_y,$$

$$\ddot{z} = -n^2z + a_z$$

Stacking the position and velocity elements into a single vector as $s := [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z}]^\top$ gives the linear time invariant (LTI) form:

$$\dot{s} = F s + G a,$$

$$F = \begin{bmatrix} \mathbf{0}_{3 \times 3} & I_3 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix}, \quad G = \begin{bmatrix} \mathbf{0}_{3 \times 3} \\ I_3 \end{bmatrix}$$

With a sample time Δt , we do a zero order hold discretization as:

$$A_d = e^{F \Delta t},$$

$$B_d = \int_0^{\Delta t} e^{F \tau} G d\tau$$

From which we obtain our final dynamics evolution as:

$$s_{k+1} = A_d s_k + B_d u_k, \quad A_d \in \mathbb{R}^{6 \times 6}, \quad B_d \in \mathbb{R}^{6 \times 3}$$

Do note that HCW is globally Linear Time Invariant (LTI) here only under certain assumptions, which are sufficient given the scope of this research: that the chief agent is following a circular orbit, we're treating with small relative distances relative to the celestial object being orbited $\|(x, y, z)\| \ll r_0$, and that the control accelerations exerted by the agents are small relative to the central gravitational field.

3.4.2 Elliptical LTV orbital relative dynamics - Tschauner and Hempel (1965)

For rollouts only, we have an elliptical dynamics model in order to study how our trained models behave under linear time variant dynamics. This means that instead of the A and B matrices remaining statics as they do with the HCW dynamics model, these A and B matrices change and evolve over time.

For this model, we again represent each agent in the chief's local frame. The main difference is that this time around we won't assume that the orbit is

perfectly circular as we did in the HCW dynamics model. Instead, the chief follows an unperturbed Keplerian elliptical orbit with gravitational parameter μ , semi-major axis a , eccentricity e , inclination i , right ascension of the ascending node Ω , argument of periapsis ω , and initial true anomaly ν_0 .

The mean motion of the chief orbit is defined as:

$$n = \sqrt{\mu/a^3}$$

At a given time $t_k = k\Delta t$, the chief mean anomaly propagates as:

$$M_k = M_0 + nt_k$$

where M_0 is obtained from ν_0 . The eccentric anomaly E_k is calculated from Kepler's equation:

$$E_k - e \sin E_k = M_k$$

and the corresponding true anomaly from the above is:

$$\nu_k = 2 \tan^{-1} \left(\frac{\sqrt{1+e} \sin(E_k/2)}{\sqrt{1-e} \cos(E_k/2)} \right)$$

The chief inertial position and velocity are then computed from its corresponding orbital elements. By defining:

$$p = a(1 - e^2), \quad r_k = \frac{p}{1 + e \cos \nu_k}$$

the chief state then becomes:

$$r_{c,p} = \begin{bmatrix} r_k \cos \nu_k \\ r_k \sin \nu_k \\ 0 \end{bmatrix}, \quad v_{c,p} = \sqrt{\frac{\mu}{p}} \begin{bmatrix} -\sin \nu_k \\ e + \cos \nu_k \\ 0 \end{bmatrix}$$

Then by using using the rotation matrix:

$$Q = R_3(\Omega)R_1(i)R_3(\omega)$$

we obtain the inertial chief state:

$$r_{c,k} = Qr_{c,p}, \quad v_{c,k} = Qv_{c,p}$$

At each time step, the chief's Radial-Transverse-Normal (RTN) frame is defined as:

$$\hat{R}_k = \frac{r_{c,k}}{\|r_{c,k}\|}, \quad \hat{N}_k = \frac{r_{c,k} \times v_{c,k}}{\|r_{c,k} \times v_{c,k}\|}, \quad \hat{T}_k = \hat{N}_k \times \hat{R}_k$$

Here, we let the concatenated matrix:

$$C_{RTN \rightarrow I,k} = \begin{bmatrix} \hat{R}_k & \hat{T}_k & \hat{N}_k \end{bmatrix}$$

map RTN coordinates into inertial coordinates. The angular velocity of the RTN frame is then:

$$\omega_k = \begin{bmatrix} 0 \\ 0 \\ \|r_{c,k} \times v_{c,k}\| / \|r_{c,k}\|^2 \end{bmatrix}$$

Now, let the relative RTN position and velocity be

$$\rho = \begin{bmatrix} \rho_R \\ \rho_T \\ \rho_N \end{bmatrix}, \quad \dot{\rho} = \begin{bmatrix} \dot{\rho}_R \\ \dot{\rho}_T \\ \dot{\rho}_N \end{bmatrix}$$

and then define the stacked state of the agent as:

$$s := \begin{bmatrix} \rho_R & \rho_T & \rho_N & \dot{\rho}_R & \dot{\rho}_T & \dot{\rho}_N \end{bmatrix}^\top$$

For a given control acceleration $u_k = (u_R, u_T, u_N)^\top$ expressed in RTN coordinates, the inertial state of the deputy at any given time t_k is:

$$r_{d,k} = r_{c,k} + C_{RTN \rightarrow I,k} \rho_k,$$

$$v_{d,k} = v_{c,k} + C_{RTN \rightarrow I,k} (\dot{\rho}_k + \omega_k \times \rho_k)$$

The deputy is then propagated over a single sample interval by using two-body inertial dynamics:

$$\dot{r}_d = v_d, \quad \dot{v}_d = -\mu \frac{r_d}{\|r_d\|^3} + C_{RTN \rightarrow I, k} u_k,$$

where the control acceleration held constant over the interval on a zero-order hold.

Following the propagation to t_{k+1} , the inertial deputy state is then transformed back into the new chief's RTN frame:

$$\rho_{k+1} = C_{I \rightarrow RTN, k+1} (r_{d, k+1} - r_{c, k+1}),$$

$$\dot{\rho}_{k+1} = C_{I \rightarrow RTN, k+1} (v_{d, k+1} - v_{c, k+1}) - \omega_{k+1} \times \rho_{k+1}.$$

This transformation defines a nonlinear discrete-time map:

$$s_{k+1} = f_k(s_k, u_k)$$

For the rollout model of the policies into this elliptical model, we use a linear time-varying approximation of this map about the chief trajectory. In other words, an approximation about zero relative state and zero control:

$$s_k = 0, \quad u_k = 0$$

The discrete LTV matrices are then computed on each step as:

$$A_k = \left. \frac{\partial f_k}{\partial s} \right|_{s=0, u=0}, \quad B_k = \left. \frac{\partial f_k}{\partial u} \right|_{s=0, u=0}.$$

In this implementation, these Jacobians are obtained by using centered finite differences in the form:

$$A_k[:, j] \approx \frac{f_k(\epsilon e_j, 0) - f_k(-\epsilon e_j, 0)}{2\epsilon}, \quad j = 1, \dots, 6,$$

$$B_k[:, j] \approx \frac{f_k(0, \epsilon e_j) - f_k(0, -\epsilon e_j)}{2\epsilon}, \quad j = 1, \dots, 3.$$

And finally, the rollout dynamics are therefore:

$$s_{k+1} = A_k s_k + B_k u_k, \quad A_k \in \mathbb{R}^{6 \times 6}, \quad B_k \in \mathbb{R}^{6 \times 3}.$$

which define the time evolution of the state in the same way we did with HCW, but using our new elliptical dynamics model.

Unlike HCW dynamics, however, the model is clearly not globally Linear Time Invariant (LTI) since the chief’s orbital radius, true anomaly, and RTN frame angular rate vary along the elliptical orbit. This approximation is valid for small relative separation relative to the chief’s orbital radius, control accelerations that are smaller relative to the central gravitational field, and assuming that the Keplerian chief trajectory remains unperturbed.

3.5 Estimation model - Bearing-Only Extended Kalman Filter (EKF)

An bearings-only EKF model, based on the work in Aidala (1979) which is based on the foundational piece by Kalman (1960), was picked for our state estimation architecture. This bearings-only EKF basically takes azimuth-elevation measurements (essentially acting as a basic optical navigation suite). Ergo, the measurement model is non-linear, even if the dynamics model may be linearized as is the case with HCW. Algorithm 1 describes the step of the KF measurement and update process:

Algorithm 1 Agent-Centric Bearing-Only Kalman Filter Update

- 1: **Given:** HCW dynamics (A, B) , step size Δt , measurement period m , process noise Q , measurement noise R
- 2: **Initialize the agent beliefs:**
- 3: Initialize defender belief over attacker $(\hat{x}_0^{D \rightarrow A}, P_0^{D \rightarrow A})$ and attacker belief over defender $(\hat{x}_0^{A \rightarrow D}, P_0^{A \rightarrow D})$
- 4: **Define the measurement model:**
- 5: For observer i and target j , let:

$$d_t^{i \rightarrow j} = p_t^j - p_t^i, \quad b_t^{i \rightarrow j} = R_{wb}^i \frac{d_t^{i \rightarrow j}}{\|d_t^{i \rightarrow j}\|_2}, \quad h(p_t^i, R_{wb}^i, p_t^j) = \begin{bmatrix} \text{atan2}(b_{y,t}^{i \rightarrow j}, b_{x,t}^{i \rightarrow j}) \\ \text{atan2}(b_{z,t}^{i \rightarrow j}, \sqrt{(b_{x,t}^{i \rightarrow j})^2 + (b_{y,t}^{i \rightarrow j})^2}) \end{bmatrix}$$

- 6: **for** $t = 1, \dots, T$ **do**
 - 7: **Propagate the true defender and attacker states:**
 - 8: Apply controls u_t^D, u_t^A
 - 9: **for** each observer-target pair $(i, j) \in \{(D, A), (A, D)\}$ **do**
 - 10: **Select the filter control input:**
 - 11: $\tilde{u}_t^j \leftarrow \text{action_access}$
 - 12: **Predict the belief mean and covariance:**
 - 13: $\hat{x}_{t|t-1}^{i \rightarrow j} \leftarrow A\hat{x}_{t-1|t-1}^{i \rightarrow j} + B\tilde{u}_t^j; \quad P_{t|t-1}^{i \rightarrow j} \leftarrow AP_{t-1|t-1}^{i \rightarrow j}A^\top + Q + B\Sigma_u B^\top$
 - 14: **if** $t \bmod m = 0$ **then**
 - 15: **Form the noisy bearing measurement:**
 - 16: $z_t^{i \rightarrow j} \leftarrow h(p_t^i, R_{wb}^i, p_t^j) + \nu_t, \nu_t \sim \mathcal{N}(0, R)$
 - 17: **Apply the EKF correction:**
 - 18: Linearize h , compute innovation $y_t^{i \rightarrow j} = z_t^{i \rightarrow j} - \hat{z}_{t|t-1}^{i \rightarrow j}$, update belief
 - 19: **Log the innovation and covariance diagnostics:**
 - 20: Log $\|y_t^{i \rightarrow j}\|_2^2$ and $\text{tr}(P_{t,\text{pos}}^{i \rightarrow j})$
 - 21: **end if**
 - 22: **if** belief diverges or leaves the allowed radius bound **then**
 - 23: **Repair the divergent belief:**
 - 24: Clip/reset belief and restore covariance to P_0
 - 25: **end if**
 - 26: **end for**
 - 27: **Build the agent observations from truth and belief:**
 - 28: Use true self-state and estimated opponent state
 - 29: **Compute reward and termination from the true physical state:**
 - 30: Evaluate reward and terminal conditions from the true propagated state
 - 31: **end for**
 - 32: **Return the final beliefs and belief-space observations:**
 - 33: $(\hat{x}_{T|T}^{D \rightarrow A}, P_{T|T}^{D \rightarrow A}), (\hat{x}_{T|T}^{A \rightarrow D}, P_{T|T}^{A \rightarrow D})$, and resulting belief-space agent observations
-

Note that the EKF needs the opponent’s action $\tilde{u}_t^j$ during training and rollout; If set to none, the state measurement would not be good at all, and early testing of action-less KF policies showed that it produced completely uncoherent policies in partial observability. Therefore, one of the big assumptions we make in this research is that, any agent has somehow measured their adversaries action using some other means, recovering the noiseless ground state with no noise or the ground state within a Gaussian noise profile.

While an Unscented Kalman Filter (UKF) would be more friendly to further sensor fusion within the proposed architecture, an EKF was picked given that early testing of our estimation-in-the-loop training signaled to extremely long training times (on the order of days of training time per phase) with a UKF in the loop, likely due to the high compute utilization of the unscented transform. Furthermore, EKFs are more common seen in real-life aerospace estimation architectures.

3.6 Controller model - CBF-QP velocity saturation controller

One of the later additions to this research was the implementation of a CBF-QP controller (Ames et al. (2017)) to saturate the velocities of the agents during training and rollout.

There’s a couple reasons behind this addition. First of all, it was discovered during the development of this research that, without the velocity saturation limit, the agents would become greedy and myopic with the access they had to their own control output commands. The result of this was that the agents, especially the defender, would pick up a lot of speed by using most of the control limit and then struggle to steer and control themselves during training and rollout.

One route we considered taking was to implement velocity/control saturating element in the reward function described in section 3.8, which would have been possible but we would have lost interpretability of the reward function and the mean reward return during training by combining various elements with different units into one

monolithic reward function. Generally speaking, as we mention throughout that section, we found our RL trainings to behave best the simpler and cleaner our reward shaping was.

Furthermore, we saw this as a great opportunity to show how a classical controller could fill in this gap and do this velocity saturation, setting a compute efficient constrain on the agents while allowing it the training logic remain as simple as possible.

We opted to use a CBF-QP controller for velocity saturation and, overall, it was highly performing during training and rollout right from its implementation: successfully constraining the net agent speed while not interfering on the agent’s learning. The process is described in detail in algorithm 2.

Algorithm 2 Velocity CBF-QP Saturation Filter

- 1: **Given:** nominal action u_t^{nom} , agent state (p_t, v_t) , box bounds $-u_{\max} \leq u \leq u_{\max}$, speed limit $v_{\max}$, CBF gain α , HCW mean motion n
 - 2: **Compute the barrier value:**
 - 3: $h_t \leftarrow v_{\max}^2 - \|v_t\|_2^2$
 - 4: **Compute the HCW drift in the velocity channel:**
 - 5:
$$f_v(p_t, v_t) \leftarrow \begin{bmatrix} 3n^2 p_{x,t} + 2nv_{y,t} \\ -2nv_{x,t} \\ -n^2 p_{z,t} \end{bmatrix}$$
 - 6: **Form the CBF half-space from $\dot{h}_t + \alpha h_t \geq 0$:**
 - 7:
$$a_t \leftarrow 2v_t, \quad b_t \leftarrow \alpha h_t - 2v_t^\top f_v(p_t, v_t)$$
 - 8: **Pose the box-constrained projection:**
 - 9:
$$u_t^* \leftarrow \arg \min_u \frac{1}{2} \|u - u_t^{\text{nom}}\|_2^2 \quad \text{s.t.} \quad a_t^\top u \leq b_t, \quad -u_{\max} \leq u \leq u_{\max}$$
 - 10: **Check whether the clipped nominal action is already feasible:**
 - 11: Clip u_t^{nom} to the box bounds; if the clipped action already satisfies $a_t^\top u \leq b_t$, return it
 - 12: **Construct the box corner that minimizes the half-space left-hand side:**
 - 13: Let $u_{\min}$ have components $(u_{\min})_k = -u_{\max}$ if $(a_t)_k \geq 0$ and $(u_{\min})_k = +u_{\max}$ otherwise
 - 14: **Handle the infeasible case:**
 - 15: If $a_t^\top u_{\min} > b_t$, return the least-violating box action $u_t^* \leftarrow u_{\min}$
 - 16: **Otherwise search the KKT multiplier:**
 - 17:
$$u(\lambda) \leftarrow \text{clip}(u_t^{\text{nom}} - \lambda a_t, -u_{\max}, u_{\max})$$
 - 18: **Bracket and bisect λ :**
 - 19: Increase λ until $a_t^\top u(\lambda) \leq b_t$, then bisect on λ to the desired tolerance
 - 20: **Return the projected feasible action:**
 - 21: $u_t^* \leftarrow u(\lambda)$
-

3.7 State, action, and observation.

Consider two agents, a defender which we refer to as player/agent #1 and an attacker which we refer to as player/agent #2. These agents evolve under discretized

HCW dynamics, relative to a chief orbit, with a discrete time sampling Δt . For each agent $i \in \{1, 2\}$, their state is defined as:

$$x_i = \begin{bmatrix} p_i \\ v_i \end{bmatrix} \in \mathbb{R}^{2D}, \quad x_i^+ = A_d x_i + B_d u_i, \quad u_i \in \mathbb{R}^D,$$

where $p_i, v_i \in \mathbb{R}^D$ are the agents position and velocity in the relative frame and u_i is a commanded acceleration clipped to a system actuation bound $\|u_i\|_\infty \leq u_{\max}$. With this in mind, the concatenated state for the environment is defined as $s = [x_1^\top, x_2^\top]^\top \in \mathbb{R}^{4D}$.

And finally, each policy observes a fixed, task-shaped observation vector defined as:

$$o = [(p_1 - c), (p_2 - c), (p_2 - p_1), v_1, v_2] \in \mathbb{R}^{5D}. \quad (3.1)$$

Which for the partially observable settings become the following for the attacker:

$$o_{1,EKF} = [(p_1 - c), (\tilde{p}_{2,1} - c), (\tilde{p}_{2,1} - p_1), v_1, \tilde{v}_{2,1}] \in \mathbb{R}^{5D}. \quad (3.2)$$

And the following for the defender:

$$o_{2,EKF} = [(\tilde{p}_{1,2} - c), (p_2 - c), (p_2 - \tilde{p}_{1,2}), \tilde{v}_{1,2}, v_2] \in \mathbb{R}^{5D}. \quad (3.3)$$

Where the quantities with tilde are those measured via the bearings-only EKF described in algorithm 1.

One future area for development on the partial observability setting would be the implementation of a self-estimation model from which each agent also has to estimate their own position in space (essentially using an IMU model and the sort), which would add another layer of realism to the research presented in this work. Given the already high degree of complexity of the current research, we decided to not pursue that direction in this work.

3.8 Reward shaping and termination conditions

We then gather the distances of each agent to the center and their relative distance and normalize relative to the radius of the arena R and a constant collision radius $r_{\text{collision}}$. We use these to define the unitless attacker-to-center d_1 , defender-to-center d_2 , and docking distance d_{dock} . Note that these normalizations are absolutely necessary, as they provide stability during training. Upon normalizing, these quantities become:

$$d_1 = \frac{\|p_1 - c\|^2}{R^2}, \quad d_2 = \frac{\|p_2 - c\|^2}{R^2}, \quad d_{\text{dock}} = \frac{\max(0, \|p_2 - p_1\| - r_{\text{collision}})}{R}$$

Note that unitless quantity d_{dock} is not squared unlike unitless quantities d_1 and d_2 on purpose, as during our game it is meant to be a correcting factor, and d_1 and d_2 are supposed to be the dominating terms.

Using these, we finally define our reward function for each time step, which for the defender is simply defined as:

$$r_1 = \underbrace{k_{\text{pos}} d_2}_{\text{encourage attacker away from asset}} - \underbrace{k_{\text{dock}} d_{\text{dock}}}_{\text{penalize large defender-attacker separation}} \quad (3.4)$$

And with episodic terminal rewards defined as:

- $r_1 += \beta_{\text{intercept}}$ for intercepting/docking to the attacker.
- $r_1 += \beta_{\text{wall}}$ for forcing the attacker out of the arena.
- $r_1 -= \beta_{\text{intercept}}$ for defender crashing onto the asset of interest.
- $r_1 -= \beta_{\text{wall}}$ for defender leaving the arena.

And in order to keep the game as a min-max zero sum, the attacker reward is simply.

$$r_2 = \underbrace{-r_1}_{\text{attacker reward function}}$$

So laid out this turns out to be a reward function in the form:

$$r_2 = \underbrace{-k_{\text{pos}} d_2}_{\text{encourage attacker towards asset}} + \underbrace{k_{\text{dock}} d_{\text{dock}}}_{\text{reward large defender-attacker separation}} \quad (3.5)$$

And with episodic terminal rewards defined as:

- $r_1 -= \beta_{\text{intercept}}$ for intercepting/docking to the attacker.
- $r_1 -= \beta_{\text{wall}}$ for forcing the attacker out of the arena.
- $r_1 += \beta_{\text{intercept}}$ for defender crashing onto the asset of interest.
- $r_1 += \beta_{\text{wall}}$ for defender leaving the arena.

3.8.1 On behavior cloning for partial observability

One must note that the partially observable policies corresponding to observations 3.2 and 3.3 also have access this form of a full state reward during training. This is because on the partially observable policies we want to purposefully match the measured observations to the full state reward, a process called behavior cloning, based on the work by Ross et al. (2011).

In this process, we essentially create a measurement-based "student" that must learn from these fully observable rewards, with the intent of matching the behavior of the full state rewards as best as possible despite sensor noise.

This process proves to be perhaps the simplest way to match sensor measurements to RL rewards, which is why it was chosen. There's other potential methods for training partially observable policies, such as distillation-based methods (CITATION

NEEDED). We looked into implementing these methods earlier on, but found them to be too computationally expensive, so we proceeded regular behavior cloning in this form.

3.8.2 Partial observability via teacher-student distillation

The partially observable policy is not trained by directly optimizing the reinforcement-learning reward under measurement-based observations. Instead, the current pipeline first trains a fully observable “teacher” policy using the full-state observation and the task reward, and then distills that policy into a measurement-based “student.” In this second stage, the student receives estimator-derived inputs rather than the full state. Concretely, the student is conditioned on the estimated relative state, an uncertainty representation derived from the estimator covariance, and the previously executed action. The teacher, by contrast, is queried using a full-state observation constructed from the underlying simulator state.

The student is then trained to imitate the teacher’s behavior rather than to match rewards directly. Its primary supervision is the teacher action, together with an auxiliary latent-intent target used in the recurrent distillation architecture. Thus, the partially observable policy learns to reproduce the control behavior of the fully observable teacher as closely as possible despite estimator error and sensor noise. In that sense, the method is more accurately described as policy distillation or imitation learning than as direct reward matching.

The current implementation also uses a DAgger-style data-aggregation procedure during training, so the student is exposed to trajectories generated partly by the teacher and partly by its own predictions. This was adopted to transfer the full-state policy into an estimator-backed recurrent student without re-running full reinforcement learning directly in the partially observable setting.

Chapter 4: Learning Methods

4.1 Single Phase PPO Algorithm

We collect on-policy results via the PPO algorithm, based on the foundational work by Schulman et al. (2017) and adapted to our dynamics and environment models, where the batches are a fixed length from N vectorized environments, each of length H , and hence yielding $B = NH$ transitions per update on each policy. Algorithm 3 describes the single phase PPO training with full-state observations, whereas algorithm 4 describes the modifications that are made to the latter when doing partial observability trainings with the bearings-only EKF measurements:

Algorithm 3 Single-Phase PPO Update with Full-State Observations

Require: Learning role $r \in \{\text{DEF}, \text{ATT}\}$, fixed opponent controller $\Pi_{\bar{r}}$, horizon H , vectorized environments N

Require: Full-state observation maps $\text{Obs}_r^{\text{full}}, \text{Obs}_{\bar{r}}^{\text{full}}$, dynamics $\mathcal{D}(\cdot)$, reward $\mathcal{R}(\cdot)$

Ensure: Updated policy parameters θ_r

```
1: Initialize vectorized PPO training:
2:   Initialize  $\text{VecEnv}(N)$ , configured dynamics, learning policy  $\pi_{\theta_r}$ , and value
   function  $V_{\theta_r}$ 
3: for  $u = 1, \dots, U$  do
4:   Prepare a new rollout batch:
5:     Optionally decay learning rate and anneal environment coefficients
6:     Initialize rollout buffer  $\mathcal{B}_r$ 
7:   for  $t = 1, \dots, H$  in parallel over  $n = 1, \dots, N$  do
8:     Build full-state observations:
9:        $o_t^{r,(n)} \leftarrow \text{Obs}_r^{\text{full}}(x_t^{(n)})$ 
10:       $o_t^{\bar{r},(n)} \leftarrow \text{Obs}_{\bar{r}}^{\text{full}}(x_t^{(n)})$ 
11:     Select learner and opponent actions:
12:        $a_t^{r,(n)} \sim \pi_{\theta_r}(\cdot \mid o_t^{r,(n)})$ 
13:        $a_t^{\bar{r},(n)} \leftarrow \Pi_{\bar{r}}(o_t^{\bar{r},(n)})$   $\triangleright$  rule-based or deterministic frozen RL policy
14:     Step the environment:
15:        $(o_{t+1}^{r,(n)}, r_t^{r,(n)}, d_t^{(n)}) \leftarrow \text{STEP}(a_t^{r,(n)}, a_t^{\bar{r},(n)})$ 
16:     Conceptually, this step entails:
17:        $x_{t+1}^{(n)} \leftarrow \mathcal{D}(x_t^{(n)}, a_t^{r,(n)}, a_t^{\bar{r},(n)})$   $\triangleright$  propagate the true spacecraft state
18:        $r_t^{r,(n)} \leftarrow \mathcal{R}(x_{t+1}^{(n)}, a_t^{r,(n)}, a_t^{\bar{r},(n)})$   $\triangleright$  evaluate the role-specific reward from
       the realized transition
19:        $d_t^{(n)} \leftarrow \text{Done}(x_{t+1}^{(n)})$   $\triangleright$  check terminal conditions such as docking,
       interception, or boundary violation
20:        $o_{t+1}^{r,(n)} \leftarrow \text{Obs}_r^{\text{full}}(x_{t+1}^{(n)})$   $\triangleright$  form the next observation from the updated
       true state
21:     Store the rollout transition:
22:       Store  $(o_t^{r,(n)}, a_t^{r,(n)}, \log \pi_{\theta_r}, V_{\theta_r}, r_t^{r,(n)}, d_t^{(n)})$  in  $\mathcal{B}_r$   $\triangleright$  terminated
       environments are reset internally in the vectorized implementation
23:   end for
24:   Compute returns and advantages from rewards:
25:     Bootstrap  $V_{\theta_r}(o_{H+1}^r)$ 
26:     Compute done-masked returns and GAE advantages from the stored rewards
27:   Update the policy and value function with PPO:
28:     Normalize advantages and optimize the PPO clipped surrogate objective
29:     using policy loss, value loss, and entropy regularization for  $E$  epochs
30: end for
```

Algorithm 4 KF-On Modifications to Algorithm 3

Require: Estimator $\mathcal{E}$ with belief state $\hat{x}_t$ and measurement model y_t

- 1: Run Algorithm 3 unchanged, except replace the observation/reward portion of each rollout step by the following:
 - 2: **Maintain both truth and belief:**
 - 3: The simulator propagates the true state x_t , while the policy receives observations derived from the estimator state $\hat{x}_t$
 - 4: **Build estimator-based observations:**
 - 5: $o_t^r \leftarrow \text{Obs}_r^{\text{KF}}(x_t, \hat{x}_t)$
 - 6: $o_t^{\bar{r}} \leftarrow \text{Obs}_{\bar{r}}^{\text{KF}}(x_t, \hat{x}_t) \triangleright$ Opponent-state entries are replaced by estimator beliefs when not directly observed
 - 7: **Select actions exactly as in Algorithm 3**
 - 8: **Propagate truth, then update the estimator:**
 - 9: $x_{t+1} \leftarrow \mathcal{D}(x_t, a_t^r, a_t^{\bar{r}})$
 - 10: $\hat{x}_{t+1} \leftarrow \mathcal{E}(\hat{x}_t, a_t^r, a_t^{\bar{r}}, y_{t+1})$
 - 11: **Compute reward from the true state:**
 - 12: $r_t^r \leftarrow \mathcal{R}(x_{t+1}, a_t^r, a_t^{\bar{r}}) \quad \triangleright$ Current KF-on training still uses true-state reward geometry
 - 13: **Store the estimator-based observation in the rollout buffer, then continue with bootstrap, GAE, and PPO updates exactly as in Algorithm 3**
-

4.1.1 Policy development

Given an observation o , we obtain an action $\mu(o)$ for the queried role who $\in \{\text{def}, \text{att}\}$. From there, this action is sampled from a factorized normal distribution and squashed, again, to enforce the actuation bound:

$$u_{\text{raw}} \sim \mathcal{N}(\mu(o), \text{diag}(\sigma^2)), \quad a = u_{\text{max}} \tanh(u_{\text{raw}}).$$

And finally the corrected log-probability for PPO is

$$\log \pi(a \mid o) = \log \mathcal{N}(u_{\text{raw}}; \mu, \sigma^2) - \sum_{j=1}^D \log(1 - \tanh^2(u_{\text{raw},j})).$$

The critic $V_\theta(o)$ then maps the full observation o to a scalar value.

4.1.2 Advantage Estimation and PPO Objective

We use generalized advantage estimation (GAE) in the following form:

$$\begin{aligned}\delta_t &= r_t + \gamma(1 - d_t) V(o_{t+1}) - V(o_t), \\ A_t &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \left(\prod_{k=1}^l (1 - d_{t+k}) \right) \delta_{t+l}.\end{aligned}$$

with terminal flags $d_t \in \{0, 1\}$. Letting $R_t = A_t + V(o_t)$ for a minibatch $\mathcal{B}$, the PPO maximizes the training objective:

$$\begin{aligned}\mathcal{L}_{\text{PPO}} = \mathbb{E}_{\mathcal{B}} \bigg[& \min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t) \\ & - c_v \underbrace{(V(o_t) - R_t)^2}_{\text{(with value clipping)}} + c_{\mathcal{H}} \mathcal{H}[\pi(\cdot | o_t)] \bigg],\end{aligned}$$

where $r_t = \exp(\log \pi_{\theta}(a_t | o_t) - \log \pi_{\text{old}}(a_t | o_t))$, ϵ is the policy clip, and $c_v, c_{\mathcal{H}} > 0$ weight the value loss and entropy bonus, while applying global gradient clipping and standard Adam optimizers.

4.2 Phased Stationary Training Schedule

The process described in algorithm 3 (plus the KF specifics from 4) is then replicated in independent phases of training. Following a 0th phase where the defender is trained against a rule-based attacker to warm-start the training, each attacker or defender policy is trained against a combination of the resulting trained defender or attacker policies from previous phases. This process is called domain randomization, where training the agents against different versions of their opponents allows for the progressive development of sequentially stronger and smarter defender and attacker policies. By training against multiple policies at once, we mitigate overfitting to any one policy, one of the core issues of RL algorithms as described in the introduction, and create smarter, more generalizable strategies for the attacker and defender. Algorithm 5 describes the phased training schedule.

Algorithm 5 Phased Alternating Training Cycle with Opponent Randomization

- 1: **Phase 0: Train defender teacher $\pi_{def,0}$ against the rule-based attacker:**
 - 2: Build cfg with `attacker_mode = RULE` and `train_role = DEF`
 - 3: Do not use a checkpoint-based opponent mixture in this phase
 - 4: Train the defender teacher and retain checkpoint θ_{D0}
 - 5: **Phase 1: Train attacker teacher $\pi_{att,1}$ against a frozen defender mixture based on $\pi_{def,0}$:**
 - 6: Build cfg with `attacker_mode = RL` and `train_role = ATT`
 - 7: Load θ_{D0} and freeze defender parameters
 - 8: Define opponent variants from the same frozen defender checkpoint:
 full: (`action_scale`, `noise_std`) = (1.0, 0.0)
 weak: (0.25, 0.05)
 very-weak: (0.0, 0.0)
 - 9: Set
 $\text{opp_mix} \leftarrow \{(\pi_{def,0}^{\text{full}}, 0.5), (\pi_{def,0}^{\text{weak}}, 0.3), (\pi_{def,0}^{\text{very-weak}}, 0.2)\}$
 - 10: Resample the active opponent variant from `opp_mix` at each episode
 - 11: Train the attacker teacher and retain checkpoint θ_{A1}
 - 12: **Phase 2: Fine-tune defender teacher $\pi_{def,0} \rightarrow \pi_{def,1}$ against a frozen attacker mixture based on $\pi_{att,1}$:**
 - 13: Build cfg with `attacker_mode = RL` and `train_role = DEF`
 - 14: Load θ_{D0} as the defender initialization
 - 15: Load θ_{A1} and freeze attacker parameters
 - 16: Define opponent variants from the same frozen attacker checkpoint:
 full: (`action_scale`, `noise_std`) = (1.0, 0.0)
 weak: (0.25, 0.05)
 very-weak: (0.0, 0.0)
 - 17: Set
 $\text{opp_mix} \leftarrow \{(\pi_{att,1}^{\text{full}}, 0.5), (\pi_{att,1}^{\text{weak}}, 0.3), (\pi_{att,1}^{\text{very-weak}}, 0.2)\}$
 - 18: Resample the active opponent variant from `opp_mix` at each episode
 - 19: Train the defender teacher and retain checkpoint θ_{D1}
 - 20: **Continue the alternating procedure for subsequent phases:**
 - 21: Repeat the same attacker-training and defender-training pattern with the most recent frozen opponent checkpoint(s) and the corresponding checkpoint-based opponent mixture until the final scheduled phase is reached (Phase 3 trains $\pi_{att,2}$ and Phase 4 trains $\pi_{def,2}$, yielding five phases total including Phase 0)
-

Most importantly though, by freezing the opposing policy on each training phase we ensure that the stationary of the Markov chain in the decision process during the learning. Other published methods on zero-sum games attempt to skip this phased

approach in favor of a simultaneous self-play method, where both agents train their policies at once and they learn within the same phase using entropy regularization methods (such as that of Rudolph et al. (2025)), but these methods tend to be unstable in certain contexts. In fact, we unsuccessfully attempted to do a similar simultaneous self-play approach in this context in the earlier stages of this research, and unfavorable results were what led us to pursue this phased stationary method.

Other approaches that we experimented with were the use of residual networks with Nash differentials or classical controller priors, which could be a potential future direction for this work. Nevertheless, we found that skipping residual networks and letting giving full power to the PPO to carry on with the optimization ended up being much simpler to iterate on and interpret results for.

4.2.1 Rule-Based Attacker (Only used for phase 0 defender warm start)

Hence, the attacker control input in the initial phase (phase 0) is not defined by a PPO policy, but rather by a saturated convex combination with predefined weights, in order to warm start the training:

$$u_2 = \text{sat}_{u_{\max}}(w_c u_{\text{center}} + w_r u_{\text{repulse}} - w_d v_2),$$

where u_{center} is a one-step ridge least-squares action that pulls p_2 towards c . Considering:

- The position projection: $P = [I \ 0] \in \mathbb{R}^{D \times 2D}$
- The next-step position error is $E_2 := PA_d x_2 - c$, (given $x_2 = \begin{bmatrix} p_2 \\ v_2 \end{bmatrix}$)
- The position influence matrix $F := PB_d \in \mathbb{R}^{D \times D}$

The least squares solution for u_{center} takes the form:

$$u_{\text{center}} = -K E_2, \quad K = (F^\top F + \lambda I)^{-1} F^\top$$

and u_{repulse} , considering $r := p_2 - p_1$, $r_{\min} > 0$, gain $g > 0$, and a small term $\varepsilon > 0$ that prevents the expression from blowing up at 0 pushes away from the defender under the definition:

$$u_{\text{repulse}} = \frac{g}{\left(\max\{r_{\min}^2, \|r\|^2\} + \varepsilon\right)^{3/2}} r.$$

4.3 Inference.

At inference during policy evaluation, we translate observations into actuator commands that can be used by the spacecraft relative to a bounded commanded acceleration. at eval time we use the same 5D observation vectors as described in 3.1, 3.2, or 3.3 depending on the case.

By default we run a deterministic evaluation, $u_t^{\text{raw}} = \mu_\theta(o_t)$, where actions come from a squashed-Gaussian actor to guarantee that $\|a\|_\infty \leq u_{\max}$. . When we report stochastic results, we sample from the diagonal Gaussian $u_t^{\text{cmd}} = u_{\max} \tanh(u_t^{\text{raw}})$, and then apply tanh squashing and clipping as $u_t^{\text{cmd}} = u_{\max} \tanh(u_t^{\text{raw}})$, which enforces a bounded policy output on the final commanded acceleration/action.

This commanded acceleration then passes through the CBF-QP velocity filter.

Consequently, the spacecraft state evolves under the realized post-filter, post-actuation acceleration rather than directly under the unsquashed or pre-filter policy output.

Algorithm 6 describes the policy inference process.

Algorithm 6 Policy Interface + Spacecraft Actuation: Policy Action $\rightarrow$ Applied Acceleration

Require: Current spacecraft state x_t , policy π_θ , max accel $u_{\max}$, dynamics $\mathcal{D}(\cdot)$

Require: Evaluation mode: deterministic or stochastic

Ensure: Applied control u_t^{app} and next state x_{t+1}

```

1: Build the current observation:
2:    $o_t \leftarrow [(p_1 - c), (p_2^{\text{obs}} - c), (p_2^{\text{obs}} - p_1), v_1, v_2^{\text{obs}}]$   $\triangleright p_2^{\text{obs}}, v_2^{\text{obs}}$  are belief states if the
   estimator is enabled
3: Select a policy action in raw (unsquashed) space:
4: if deterministic evaluation then
5:    $u_t^{raw} \leftarrow \mu_\theta(o_t)$ 
6: else
7:    $u_t^{raw} \sim \mathcal{N}(\mu_\theta(o_t), \text{diag}(\sigma_\theta^2(o_t)))$ 
8: end if
9: Squash + scale (bounded command):
10:   $u_t^{cmd} \leftarrow u_{\max} \tanh(u_t^{raw})$   $\triangleright u_t^{cmd} \in [-u_{\max}, u_{\max}]^D$ 
11: Apply the velocity saturation filter:
12: if velocity filter enabled then
13:    $\tilde{u}_t \leftarrow \mathcal{S}(x_t, u_t^{cmd})$   $\triangleright$  velocity-CBF filtered command
14: else
15:    $\tilde{u}_t \leftarrow u_t^{cmd}$ 
16: end if
17: Enforce actuator bounds:
18:   $u_t^{app} \leftarrow \text{clip}(\tilde{u}_t, -u_{\max}, +u_{\max})$ 
19: Propagate spacecraft dynamics:
20:   $x_{t+1} \leftarrow \mathcal{D}(x_t, u_t^{app})$   $\triangleright$  e.g. HCW:  $x_{t+1} = A_d x_t + B_d u_t^{app}$ 
21: return ( $u_t^{app}, x_{t+1}$ )

```

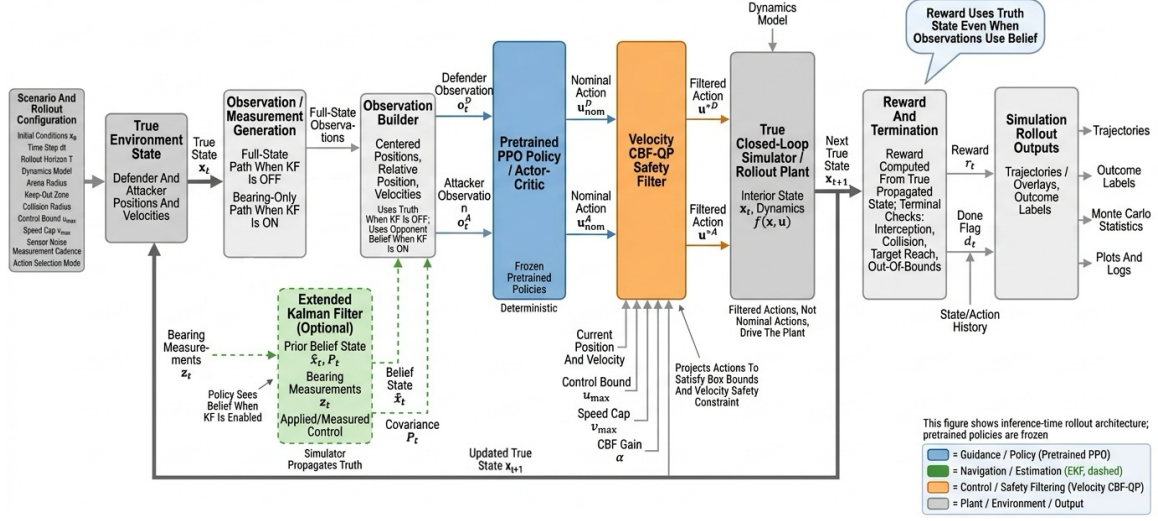


Figure 4.1: Diagram showing the Inference

4.4 Metrics and Evaluation

For a rollout of length T , we report

$$d_2(T), \quad \text{median}_t d_1(t), \quad \text{median}_t r_{\text{rel}}^2(t), \quad \Delta d_2(t) = d_2(t+1) - d_2(t),$$

to capture attacker proximity to center, defender standoff, agent separation, and monotonic progress to center.

4.5 Metrics and Evaluation

We evaluate each trained policy pair using Monte Carlo rollouts over a fixed test horizon T , with randomized initial conditions drawn from the corresponding evaluation arena. For each experiment, we report both *outcome metrics* and *trajectory-level metrics*.

Primary outcome metrics. The primary evaluation metric is the defender success rate, defined as the fraction of rollouts in which the defender wins by either capturing the attacker or forcing the attacker out of bounds. We also report the attacker success rate, decomposed by terminal mode, together with the timeout rate. For each outcome category, we report the count and empirical percentage across all Monte Carlo trials, together with the standard error of the percentage.

Trajectory-level metrics. To summarize the geometry and control behavior along a rollout, we report

$$d_2(T), \quad \text{median}_t d_1(t), \quad \text{median}_t r_{\text{rel}}^2(t), \quad \Delta d_2(t) = d_2(t+1) - d_2(t),$$

where $d_2(T)$ measures the attacker proximity to the asset at the terminal time, $\text{median}_t d_1(t)$ summarizes the defender stand-off from the asset, $\text{median}_t r_{\text{rel}}^2(t)$ summarizes the typical defender-attacker separation, and $\Delta d_2(t)$ measures step-to-step progress of the attacker toward the asset.

Control-effort and runtime metrics. In addition, we report total control effort through cumulative Δv expenditure for both agents, summarized by the median and interquartile range $[Q_1, Q_3]$ across rollouts. We also report rollout step time in milliseconds in order to quantify inference-time computational cost.

Comparison axes. All metrics are compared across: (i) HCW versus elliptic LTV rollouts, (ii) full-state versus EKF-based observations, (iii) different arena sizes, and (iv) different defender-attacker policy pairings across training phases. This allows us to assess nominal performance, robustness to model mismatch, robustness to partial observability, and generalization across evaluation regimes.

Chapter 5: Training Experiments

We did our trainings on our own NVIDIA GeForce GPUs available in the CDUS lab, we then let the training run while randomizing the starting states (positions and velocities). Initial policies were trained with relatively myopic policies ($\gamma = 0.991$), actuation bounds of ± 2 N, and a capture radius of 0.75 meters.

We performed the training of our policies in our own NVIDIA GeForce GPUs available in the CDUS lab, we then let the training run while randomizing the starting states (positions and velocities). Initial policies were trained with relatively myopic policies ($\gamma = 0.991$), actuation bounds of ± 2 N, and a capture radius of 0.75 meters. We tested on a number of different hyperparameters before converging on these values.

Overall, just from analyzing each curve described in figure ?? we see that the defender manages to produce strong, convergent policies across the multiple phases of training, and that the partially observable pipeline follows the same pattern. However, we see that the attacker progressively starts to struggle to deliver coherent policies across the phases, leading it to deliver a completely incoherent policy $\pi_{\text{att},2}$. Since our focus on this project is to deliver the strongest possible defender policies, this is good news as it signals that once the defender is progressively stronger and more generalizable thanks to the opponent randomization strategy that was described before, allowing it to be resilient throughout the progression of the training.

5.1 Trainings Presented on this Thesis

5.2 Training case 1: $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$

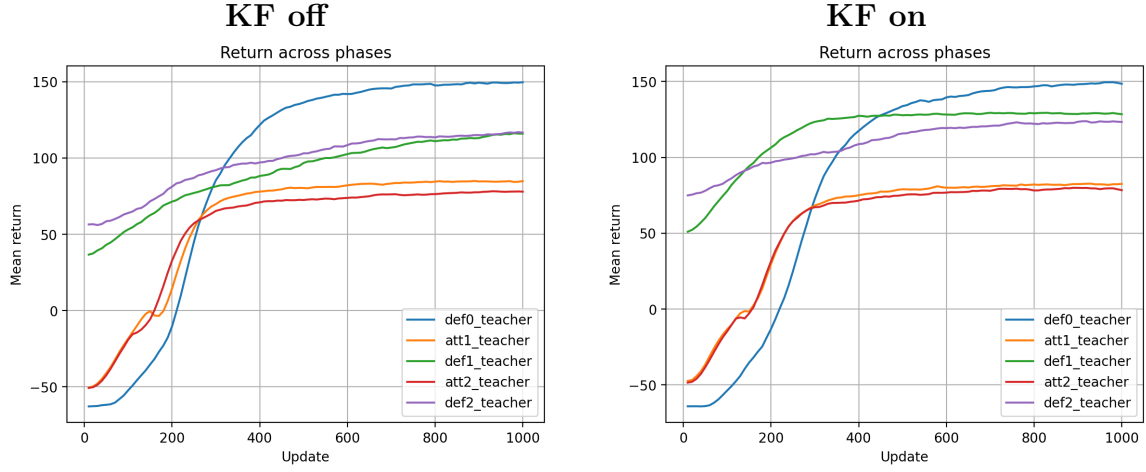


Figure 5.1: Combined training return curves for $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$.

5.3 Training case: $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$

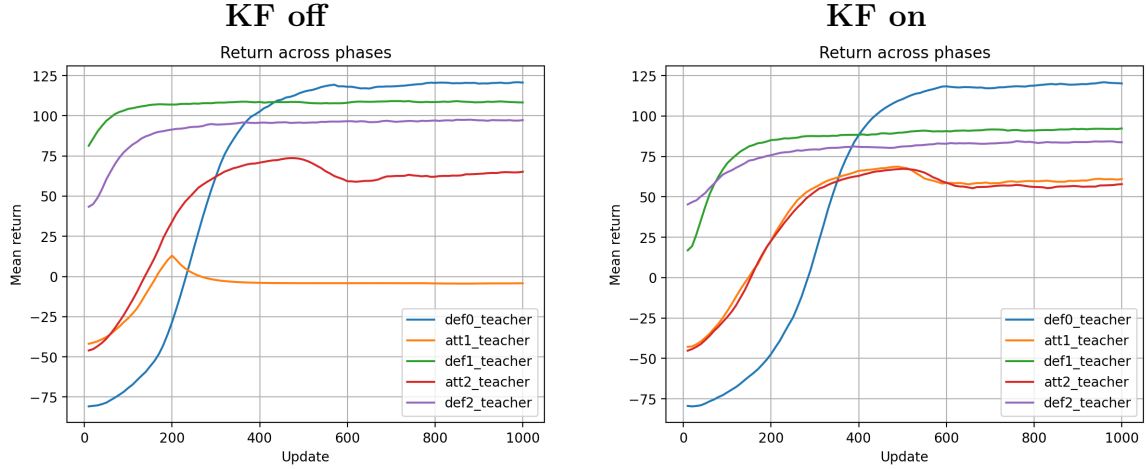


Figure 5.2: Combined training return curves for $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$.

5.4 Training case: $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$

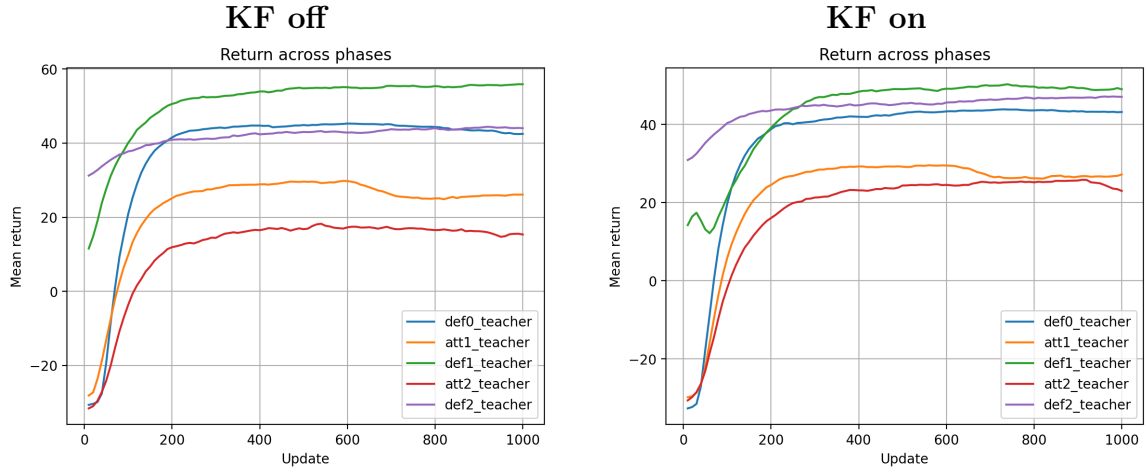


Figure 5.3: Combined training return curves for $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$.

5.5 Sample Rollout Explained in Images

Chapter 6: Policy Monte Carlo Testing

To run an experiment to test the validity of these policies, we use a Monte Carlo (MC) based method to evaluate the performance of the already trained policies. This way we can run a large amount of simulations on these policies while dispersing different initial states of the agents, namely their starting positions and velocities within the boundaries of the training arena. By rolling the simulations across a large number of iterations (4800 to be precise) and storing the outcome of each simulation index, we can obtain statistically significant results that can tell us about the performance of the policies to do their assigned tasks. Figure ?? shows starting positions were used in testing for all three initial phases in 3-D space.

6.1 Training case: $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$

6.1.1 HCW

6.1.1.1 20 meter radius arena

Table 6.1: Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1272; 26.5 $\pm$ 0.6%	4216; 87.8 $\pm$ 0.5%	852; 17.8 $\pm$ 0.6%	4441; 92.5 $\pm$ 0.4%
Att. wall	18; 0.4 $\pm$ 0.1%	15; 0.3 $\pm$ 0.1%	0; 0.0 $\pm$ 0.0%	4; 0.1 $\pm$ 0.0%
<i>Attacker wins:</i>				
Att. hit OI	3367; 70.1 $\pm$ 0.7%	382; 8.0 $\pm$ 0.4%	3808; 79.3 $\pm$ 0.6%	224; 4.7 $\pm$ 0.3%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	14; 0.3 $\pm$ 0.1%	11; 0.2 $\pm$ 0.1%	13; 0.3 $\pm$ 0.1%	1; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	129; 2.7 $\pm$ 0.2%	176; 3.7 $\pm$ 0.3%	127; 2.6 $\pm$ 0.2%	130; 2.7 $\pm$ 0.2%
<i>Delta-v statistics:</i>				
Def. total Δv	3.50 [1.90, 4.19] m/s	1.73 [1.54, 1.97] m/s	3.96 [3.26, 4.35] m/s	1.78 [1.59, 2.00] m/s
Att. total Δv	3.22 [1.84, 3.84] m/s	1.71 [1.49, 1.97] m/s	3.64 [3.02, 4.00] m/s	1.67 [1.48, 1.88] m/s
Total n	4800			
Step time	0.0895 [0.0752, 0.1145] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1875; 39.1 $\pm$ 0.7%	4580; 95.4 $\pm$ 0.3%	1731; 36.1 $\pm$ 0.7%	4399; 91.6 $\pm$ 0.4%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	13; 0.3 $\pm$ 0.1%	3; 0.1 $\pm$ 0.0%
<i>Attacker wins:</i>				
Att. hit OI	2729; 56.9 $\pm$ 0.7%	98; 2.0 $\pm$ 0.2%	2847; 59.3 $\pm$ 0.7%	258; 5.4 $\pm$ 0.3%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	2; 0.0 $\pm$ 0.0%	4; 0.1 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	196; 4.1 $\pm$ 0.3%	122; 2.5 $\pm$ 0.2%	207; 4.3 $\pm$ 0.3%	136; 2.8 $\pm$ 0.2%
<i>Delta-v statistics:</i>				
Def. total Δv	2.71 [1.59, 3.70] m/s	1.62 [1.46, 1.82] m/s	3.66 [1.69, 4.45] m/s	1.76 [1.54, 2.02] m/s
Att. total Δv	2.40 [1.49, 3.38] m/s	1.56 [1.37, 1.78] m/s	3.27 [1.59, 3.97] m/s	1.65 [1.41, 1.96] m/s
Total n	4800			
Step time	3.7445 [3.7192, 3.7721] ms			

6.1.1.2 100 meter radius arena

Table 6.2: Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1163; $12.1 \pm 0.3\%$	6632; $69.1 \pm 0.5\%$	1876; $19.5 \pm 0.4\%$	5480; $57.1 \pm 0.5\%$
Att. wall	36; $0.4 \pm 0.1\%$	27; $0.3 \pm 0.1\%$	269; $2.8 \pm 0.2\%$	257; $2.7 \pm 0.2\%$
<i>Attacker wins:</i>				
Att. hit OI	8350; $87.0 \pm 0.3\%$	2923; $30.4 \pm 0.5\%$	7390; $77.0 \pm 0.4\%$	3813; $39.7 \pm 0.5\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	1; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
<i>Ties:</i>				
Timeout	51; $0.5 \pm 0.1\%$	18; $0.2 \pm 0.0\%$	64; $0.7 \pm 0.1\%$	50; $0.5 \pm 0.1\%$
<i>Delta-v statistics:</i>				
Def. total Δv	6.35 [5.21, 7.46] m/s	4.40 [3.47, 6.33] m/s	6.52 [5.08, 7.68] m/s	4.96 [3.53, 7.22] m/s
Att. total Δv	5.33 [4.33, 6.45] m/s	4.26 [3.27, 5.77] m/s	5.46 [4.22, 6.81] m/s	4.38 [3.02, 6.32] m/s
Total n	9600			
Step time	0.1032 [0.0803, 0.1419] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	2825; $29.4 \pm 0.5\%$	6156; $64.1 \pm 0.5\%$	2338; $24.4 \pm 0.4\%$	6527; $68.0 \pm 0.5\%$
Att. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	41; $0.4 \pm 0.1\%$	39; $0.4 \pm 0.1\%$
<i>Attacker wins:</i>				
Att. hit OI	6710; $69.9 \pm 0.5\%$	3408; $35.5 \pm 0.5\%$	7157; $74.6 \pm 0.4\%$	2995; $31.2 \pm 0.5\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
<i>Ties:</i>				
Timeout	65; $0.7 \pm 0.1\%$	36; $0.4 \pm 0.1\%$	64; $0.7 \pm 0.1\%$	39; $0.4 \pm 0.1\%$
<i>Delta-v statistics:</i>				
Def. total Δv	5.80 [4.49, 7.06] m/s	4.45 [3.39, 6.02] m/s	6.28 [4.89, 7.54] m/s	4.36 [3.44, 6.01] m/s
Att. total Δv	4.52 [3.55, 5.59] m/s	3.74 [2.86, 4.83] m/s	4.83 [3.64, 6.05] m/s	3.71 [2.75, 4.98] m/s
Total n	9600			
Step time	3.8276 [3.7969, 3.8537] ms			

6.1.2 Elliptic LTV

6.1.2.1 20 meter radius arena

Table 6.3: Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1268; $26.4 \pm 0.6\%$	4208; $87.7 \pm 0.5\%$	862; $18.0 \pm 0.6\%$	4453; $92.8 \pm 0.4\%$
Att. wall	18; $0.4 \pm 0.1\%$	15; $0.3 \pm 0.1\%$	0; $0.0 \pm 0.0\%$	4; $0.1 \pm 0.0\%$
<i>Attacker wins:</i>				
Att. hit OI	3372; $70.2 \pm 0.7\%$	390; $8.1 \pm 0.4\%$	3799; $79.1 \pm 0.6\%$	212; $4.4 \pm 0.3\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	14; $0.3 \pm 0.1\%$	11; $0.2 \pm 0.1\%$	12; $0.2 \pm 0.1\%$	1; $0.0 \pm 0.0\%$
<i>Ties:</i>				
Timeout	128; $2.7 \pm 0.2\%$	176; $3.7 \pm 0.3\%$	127; $2.6 \pm 0.2\%$	130; $2.7 \pm 0.2\%$
<i>Delta-v statistics:</i>				
Def. total Δv	3.50 [1.90, 4.19] m/s	1.73 [1.54, 1.97] m/s	3.95 [3.25, 4.34] m/s	1.78 [1.59, 1.99] m/s
Att. total Δv	3.22 [1.84, 3.84] m/s	1.71 [1.49, 1.97] m/s	3.64 [3.01, 4.00] m/s	1.67 [1.48, 1.88] m/s
Total n	4800			
Step time	0.3076 [0.2641, 0.3868] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<i>Attacker wins:</i>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<i>Ties:</i>				
Timeout	N/A	N/A	N/A	N/A
<i>Delta-v statistics:</i>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.1.2.2 100 meter radius arena

Table 6.4: Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 0.1$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 0.05$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].
KF off

Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1151; $12.0 \pm 0.3\%$	6616; $68.9 \pm 0.5\%$	1903; $19.8 \pm 0.4\%$	5512; $57.4 \pm 0.5\%$
Att. wall	36; $0.4 \pm 0.1\%$	27; $0.3 \pm 0.1\%$	269; $2.8 \pm 0.2\%$	256; $2.7 \pm 0.2\%$
<i>Attacker wins:</i>				
Att. hit OI	8363; $87.1 \pm 0.3\%$	2940; $30.6 \pm 0.5\%$	7363; $76.7 \pm 0.4\%$	3783; $39.4 \pm 0.5\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
<i>Ties:</i>				
Timeout	50; $0.5 \pm 0.1\%$	17; $0.2 \pm 0.0\%$	65; $0.7 \pm 0.1\%$	49; $0.5 \pm 0.1\%$
<i>Delta-v statistics:</i>				
Def. total Δv	6.35 [5.22, 7.48] m/s	4.40 [3.48, 6.35] m/s	6.51 [5.06, 7.68] m/s	4.94 [3.52, 7.20] m/s
Att. total Δv	5.34 [4.34, 6.47] m/s	4.27 [3.27, 5.79] m/s	5.46 [4.20, 6.80] m/s	4.36 [3.01, 6.31] m/s
Total n	9600			
Step time	0.1550 [0.1217, 0.2119] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<i>Attacker wins:</i>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<i>Ties:</i>				
Timeout	N/A	N/A	N/A	N/A
<i>Delta-v statistics:</i>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.2 Training case: $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$

6.2.1 HCW

6.2.1.1 20 meter radius arena

Table 6.5: Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	4117; 85.8 $\pm$ 0.5%	4745; 98.9 $\pm$ 0.2%	1698; 35.4 $\pm$ 0.7%	4522; 94.2 $\pm$ 0.3%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	19; 0.4 $\pm$ 0.1%	29; 0.6 $\pm$ 0.1%
<i>Attacker wins:</i>				
Att. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	2865; 59.7 $\pm$ 0.7%	42; 0.9 $\pm$ 0.1%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	683; 14.2 $\pm$ 0.5%	55; 1.1 $\pm$ 0.2%	218; 4.5 $\pm$ 0.3%	207; 4.3 $\pm$ 0.3%
<i>Delta-v statistics:</i>				
Def. total Δv	4.00 [3.33, 11.22] m/s	3.36 [2.85, 3.91] m/s	7.30 [4.78, 9.13] m/s	4.75 [3.98, 5.52] m/s
Att. total Δv	6.30 [4.98, 52.19] m/s	5.92 [4.75, 7.27] m/s	6.10 [4.05, 8.04] m/s	4.32 [3.39, 5.25] m/s
Total n	4800			
Step time	0.7695 [0.1514, 0.9584] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	817; 17.0 $\pm$ 0.5%	4632; 96.5 $\pm$ 0.3%	1263; 26.3 $\pm$ 0.6%	4466; 93.0 $\pm$ 0.4%
Att. wall	13; 0.3 $\pm$ 0.1%	1; 0.0 $\pm$ 0.0%	106; 2.2 $\pm$ 0.2%	100; 2.1 $\pm$ 0.2%
<i>Attacker wins:</i>				
Att. hit OI	3830; 79.8 $\pm$ 0.6%	12; 0.2 $\pm$ 0.1%	3201; 66.7 $\pm$ 0.7%	40; 0.8 $\pm$ 0.1%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	140; 2.9 $\pm$ 0.2%	155; 3.2 $\pm$ 0.3%	230; 4.8 $\pm$ 0.3%	194; 4.0 $\pm$ 0.3%
<i>Delta-v statistics:</i>				
Def. total Δv	8.44 [7.10, 9.45] m/s	4.80 [4.06, 5.54] m/s	9.25 [6.00, 10.55] m/s	5.39 [4.58, 6.24] m/s
Att. total Δv	7.65 [6.06, 8.92] m/s	4.30 [3.43, 5.24] m/s	8.11 [5.37, 9.45] m/s	4.92 [3.92, 6.02] m/s
Total n	4800			
Step time	3.8639 [3.8376, 3.8927] ms			

6.2.1.2 100 meter radius arena

Table 6.6: Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	5927; 61.7 $\pm$ 0.5%	3865; 40.3 $\pm$ 0.5%	1805; 18.8 $\pm$ 0.4%	6895; 71.8 $\pm$ 0.5%
Att. wall	593; 6.2 $\pm$ 0.2%	5622; 58.6 $\pm$ 0.5%	5; 0.1 $\pm$ 0.0%	9; 0.1 $\pm$ 0.0%
<i>Attacker wins:</i>				
Att. hit OI	2; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	7730; 80.5 $\pm$ 0.4%	2660; 27.7 $\pm$ 0.5%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	110; 1.1 $\pm$ 0.1%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	3078; 32.1 $\pm$ 0.5%	3; 0.0 $\pm$ 0.0%	60; 0.6 $\pm$ 0.1%	36; 0.4 $\pm$ 0.1%
<i>Delta-v statistics:</i>				
Def. total Δv	20.65 [12.73, 96.46] m/s	7.64 [5.87, 9.74] m/s	12.33 [9.92, 14.79] m/s	10.27 [8.19, 13.41] m/s
Att. total Δv	89.89 [10.38, 477.25] m/s	7.05 [5.00, 12.46] m/s	10.62 [8.32, 13.01] m/s	8.80 [6.84, 11.84] m/s
Total n	9600			
Step time	0.1115 [0.0635, 0.1906] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1911; 19.9 $\pm$ 0.4%	7569; 78.8 $\pm$ 0.4%	1065; 11.1 $\pm$ 0.3%	6073; 63.3 $\pm$ 0.5%
Att. wall	0; 0.0 $\pm$ 0.0%	1; 0.0 $\pm$ 0.0%	76; 0.8 $\pm$ 0.1%	39; 0.4 $\pm$ 0.1%
<i>Attacker wins:</i>				
Att. hit OI	7673; 79.9 $\pm$ 0.4%	2000; 20.8 $\pm$ 0.4%	8409; 87.6 $\pm$ 0.3%	3449; 35.9 $\pm$ 0.5%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	16; 0.2 $\pm$ 0.0%	30; 0.3 $\pm$ 0.1%	50; 0.5 $\pm$ 0.1%	39; 0.4 $\pm$ 0.1%
<i>Delta-v statistics:</i>				
Def. total Δv	13.43 [11.13, 15.90] m/s	9.90 [8.16, 12.23] m/s	15.47 [12.91, 18.18] m/s	11.97 [9.47, 15.34] m/s
Att. total Δv	10.87 [8.76, 13.10] m/s	7.83 [6.15, 9.86] m/s	12.45 [10.32, 14.74] m/s	9.85 [7.78, 12.56] m/s
Total n	9600			
Step time	7.7114 [7.6147, 7.7953] ms			

6.2.2 Elliptic LTV

6.2.2.1 20 meter radius arena

Table 6.7: Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<u>Defender wins:</u>				
Def. capture	4120; $85.8 \pm 0.5\%$	4746; $98.9 \pm 0.2\%$	1695; $35.3 \pm 0.7\%$	4521; $94.2 \pm 0.3\%$
Att. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	19; $0.4 \pm 0.1\%$	30; $0.6 \pm 0.1\%$
<u>Attacker wins:</u>				
Att. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	2867; $59.7 \pm 0.7\%$	43; $0.9 \pm 0.1\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
<u>Ties:</u>				
Timeout	680; $14.2 \pm 0.5\%$	54; $1.1 \pm 0.2\%$	219; $4.6 \pm 0.3\%$	206; $4.3 \pm 0.3\%$
<u>Delta-v statistics:</u>				
Def. total Δv	3.99 [3.33, 11.16] m/s	3.37 [2.85, 3.91] m/s	7.29 [4.78, 9.13] m/s	4.75 [3.98, 5.52] m/s
Att. total Δv	6.30 [5.00, 52.28] m/s	5.93 [4.76, 7.26] m/s	6.10 [4.05, 8.05] m/s	4.32 [3.39, 5.25] m/s
Total n	4800			
Step time	1.1877 [0.4099, 1.4866] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<u>Defender wins:</u>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<u>Attacker wins:</u>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<u>Ties:</u>				
Timeout	N/A	N/A	N/A	N/A
<u>Delta-v statistics:</u>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.2.2.2 100 meter radius arena

Table 6.8: Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 0.5$, $v_{\max} = 1.5$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	5918; $61.6 \pm 0.5\%$	3860; $40.2 \pm 0.5\%$	1808; $18.8 \pm 0.4\%$	6901; $71.9 \pm 0.5\%$
Att. wall	596; $6.2 \pm 0.2\%$	5628; $58.6 \pm 0.5\%$	4; $0.0 \pm 0.0\%$	9; $0.1 \pm 0.0\%$
<i>Attacker wins:</i>				
Att. hit OI	2; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	7726; $80.5 \pm 0.4\%$	2657; $27.7 \pm 0.5\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	109; $1.1 \pm 0.1\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
<i>Ties:</i>				
Timeout	3084; $32.1 \pm 0.5\%$	3; $0.0 \pm 0.0\%$	62; $0.6 \pm 0.1\%$	33; $0.3 \pm 0.1\%$
<i>Delta-v statistics:</i>				
Def. total Δv	20.67 [12.71, 96.78] m/s	7.65 [5.87, 9.75] m/s	12.32 [9.92, 14.79] m/s	10.26 [8.19, 13.39] m/s
Att. total Δv	90.30 [10.54, 477.53] m/s	7.17 [5.05, 12.72] m/s	10.62 [8.33, 13.01] m/s	8.80 [6.84, 11.84] m/s
Total n	9600			
Step time	0.1732 [0.1087, 0.2806] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<i>Attacker wins:</i>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<i>Ties:</i>				
Timeout	N/A	N/A	N/A	N/A
<i>Delta-v statistics:</i>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.3 Training case: $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$

6.3.1 HCW

6.3.1.1 20 meter radius arena

Table 6.9: Monte Carlo outcomes: HCW, 20 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1306; 27.2 $\pm$ 0.6%	4760; 99.2 $\pm$ 0.1%	1710; 35.6 $\pm$ 0.7%	4187; 87.2 $\pm$ 0.5%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	2761; 57.5 $\pm$ 0.7%	539; 11.2 $\pm$ 0.5%
<i>Attacker wins:</i>				
Att. hit OI	3389; 70.6 $\pm$ 0.7%	0; 0.0 $\pm$ 0.0%	272; 5.7 $\pm$ 0.3%	3; 0.1 $\pm$ 0.0%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	105; 2.2 $\pm$ 0.2%	40; 0.8 $\pm$ 0.1%	57; 1.2 $\pm$ 0.2%	71; 1.5 $\pm$ 0.2%
<i>Delta-v statistics:</i>				
Def. total Δv	20.26 [14.50, 26.61] m/s	3.54 [2.96, 4.10] m/s	8.39 [6.77, 11.05] m/s	9.17 [7.13, 12.23] m/s
Att. total Δv	4.68 [3.61, 5.79] m/s	2.64 [2.00, 3.30] m/s	6.95 [5.63, 8.41] m/s	4.93 [4.12, 5.89] m/s
Total n	4800			
Step time	0.0843 [0.0696, 0.1044] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1786; 37.2 $\pm$ 0.7%	4697; 97.9 $\pm$ 0.2%	3129; 65.2 $\pm$ 0.7%	4738; 98.7 $\pm$ 0.2%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Attacker wins:</i>				
Att. hit OI	2551; 53.1 $\pm$ 0.7%	21; 0.4 $\pm$ 0.1%	1224; 25.5 $\pm$ 0.6%	6; 0.1 $\pm$ 0.1%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	463; 9.6 $\pm$ 0.4%	82; 1.7 $\pm$ 0.2%	447; 9.3 $\pm$ 0.4%	56; 1.2 $\pm$ 0.2%
<i>Delta-v statistics:</i>				
Def. total Δv	17.05 [8.53, 23.85] m/s	4.33 [3.56, 5.07] m/s	6.47 [4.73, 9.03] m/s	5.33 [4.31, 6.55] m/s
Att. total Δv	4.10 [2.94, 5.31] m/s	2.78 [2.05, 3.57] m/s	4.62 [2.99, 7.46] m/s	3.45 [2.53, 4.57] m/s
Total n	4800			
Step time	3.8856 [3.8552, 3.9203] ms			

6.3.1.2 100 meter radius arena

Table 6.10: Monte Carlo outcomes: HCW, 100 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	4738; 49.4 $\pm$ 0.5%	9001; 93.8 $\pm$ 0.2%	1931; 20.1 $\pm$ 0.4%	5028; 52.4 $\pm$ 0.5%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	3145; 32.8 $\pm$ 0.5%	1577; 16.4 $\pm$ 0.4%
<i>Attacker wins:</i>				
Att. hit OI	4851; 50.5 $\pm$ 0.5%	587; 6.1 $\pm$ 0.2%	4282; 44.6 $\pm$ 0.5%	2900; 30.2 $\pm$ 0.5%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	181; 1.9 $\pm$ 0.1%	35; 0.4 $\pm$ 0.1%
<i>Ties:</i>				
Timeout	11; 0.1 $\pm$ 0.0%	12; 0.1 $\pm$ 0.0%	61; 0.6 $\pm$ 0.1%	60; 0.6 $\pm$ 0.1%
<i>Delta-v statistics:</i>				
Def. total Δv	45.11 [14.23, 86.52] m/s	7.54 [6.04, 9.32] m/s	11.70 [9.05, 15.94] m/s	14.70 [10.21, 23.49] m/s
Att. total Δv	7.80 [5.91, 10.01] m/s	4.76 [3.60, 6.00] m/s	7.45 [5.94, 9.20] m/s	6.57 [5.21, 8.31] m/s
Total n	9600			
Step time	0.0939 [0.0731, 0.1228] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	3711; 38.7 $\pm$ 0.5%	6272; 65.3 $\pm$ 0.5%	3108; 32.4 $\pm$ 0.5%	4998; 52.1 $\pm$ 0.5%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Attacker wins:</i>				
Att. hit OI	5660; 59.0 $\pm$ 0.5%	3161; 32.9 $\pm$ 0.5%	6004; 62.5 $\pm$ 0.5%	4494; 46.8 $\pm$ 0.5%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	229; 2.4 $\pm$ 0.2%	167; 1.7 $\pm$ 0.1%	488; 5.1 $\pm$ 0.2%	108; 1.1 $\pm$ 0.1%
<i>Delta-v statistics:</i>				
Def. total Δv	26.77 [8.02, 66.33] m/s	10.03 [7.61, 14.53] m/s	12.59 [9.54, 16.08] m/s	12.23 [9.19, 16.92] m/s
Att. total Δv	6.68 [4.85, 9.42] m/s	5.72 [4.37, 7.56] m/s	6.55 [5.05, 8.28] m/s	5.85 [4.34, 7.48] m/s
Total n	9600			
Step time	3.8542 [3.8296, 3.8836] ms			

6.3.2 Elliptic LTV

6.3.2.1 20 meter radius arena

Table 6.11: Monte Carlo outcomes: Elliptic LTV, 20 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	1310; 27.3 $\pm$ 0.6%	4760; 99.2 $\pm$ 0.1%	1717; 35.8 $\pm$ 0.7%	4195; 87.4 $\pm$ 0.5%
Att. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	2760; 57.5 $\pm$ 0.7%	529; 11.0 $\pm$ 0.5%
<i>Attacker wins:</i>				
Att. hit OI	3385; 70.5 $\pm$ 0.7%	0; 0.0 $\pm$ 0.0%	267; 5.6 $\pm$ 0.3%	3; 0.1 $\pm$ 0.0%
Def. hit OI	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
Def. wall	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%	0; 0.0 $\pm$ 0.0%
<i>Ties:</i>				
Timeout	105; 2.2 $\pm$ 0.2%	40; 0.8 $\pm$ 0.1%	56; 1.2 $\pm$ 0.2%	73; 1.5 $\pm$ 0.2%
<i>Delta-v statistics:</i>				
Def. total Δv	20.48 [14.76, 26.74] m/s	3.53 [2.96, 4.10] m/s	8.54 [6.85, 11.37] m/s	9.46 [7.37, 12.54] m/s
Att. total Δv	4.68 [3.60, 5.79] m/s	2.64 [2.00, 3.30] m/s	6.96 [5.66, 8.44] m/s	4.96 [4.14, 5.91] m/s
Total n	4800			
Step time	0.3433 [0.2875, 0.4207] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<i>Attacker wins:</i>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<i>Ties:</i>				
Timeout	N/A	N/A	N/A	N/A
<i>Delta-v statistics:</i>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.3.2.2 100 meter radius arena

Table 6.12: Monte Carlo outcomes: Elliptic LTV, 100 meter radius arena, $u_{\max} = 2.0$, $v_{\max} = 1$, $v_{\max}^{\text{IC}} = 1.0$. Cells: count; % $\pm$ SE. Δv and time: median [Q1, Q3].

KF off				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	4746; $49.4 \pm 0.5\%$	9007; $93.8 \pm 0.2\%$	1973; $20.6 \pm 0.4\%$	5079; $52.9 \pm 0.5\%$
Att. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	3136; $32.7 \pm 0.5\%$	1564; $16.3 \pm 0.4\%$
<i>Attacker wins:</i>				
Att. hit OI	4843; $50.4 \pm 0.5\%$	580; $6.0 \pm 0.2\%$	4245; $44.2 \pm 0.5\%$	2863; $29.8 \pm 0.5\%$
Def. hit OI	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$
Def. wall	0; $0.0 \pm 0.0\%$	0; $0.0 \pm 0.0\%$	184; $1.9 \pm 0.1\%$	34; $0.4 \pm 0.1\%$
<i>Ties:</i>				
Timeout	11; $0.1 \pm 0.0\%$	13; $0.1 \pm 0.0\%$	62; $0.6 \pm 0.1\%$	60; $0.6 \pm 0.1\%$
<i>Delta-v statistics:</i>				
Def. total Δv	45.56 [15.27, 87.53] m/s	7.55 [6.05, 9.35] m/s	12.04 [9.20, 16.82] m/s	15.50 [10.65, 24.61] m/s
Att. total Δv	7.82 [5.95, 10.02] m/s	4.76 [3.61, 6.01] m/s	7.50 [5.98, 9.28] m/s	6.61 [5.25, 8.40] m/s
Total n	9600			
Step time	0.1467 [0.1153, 0.1931] ms			
KF on				
Outcome	$\pi_{\text{def},0}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},1}$	$\pi_{\text{def},1}$ VS $\pi_{\text{att},2}$	$\pi_{\text{def},2}$ VS $\pi_{\text{att},2}$
<i>Defender wins:</i>				
Def. capture	N/A	N/A	N/A	N/A
Att. wall	N/A	N/A	N/A	N/A
<i>Attacker wins:</i>				
Att. hit OI	N/A	N/A	N/A	N/A
Def. hit OI	N/A	N/A	N/A	N/A
Def. wall	N/A	N/A	N/A	N/A
<i>Ties:</i>				
Timeout	N/A	N/A	N/A	N/A
<i>Delta-v statistics:</i>				
Def. total Δv	N/A	N/A	N/A	N/A
Att. total Δv	N/A	N/A	N/A	N/A
Total n	N/A			
Step time	N/A			

6.3.2.3 Discussion of Monte Carlo results

Note: Pattern for percentages and times cited corresponds to the following order: Fully Observable (Table ??) / Partially Observable with ground truth (Table ??) / Partially Observable with noise (Table ??).

Across tables ??, ??, and ??, Phase 0 shows us that the $\pi_{\text{def},0}$ RL defender succeeds in capturing the classical attacker in (41.8 %, 48.4 %, 99.0 %) of the runs. The attacker hits the object of interest (OI) in only (3.2 %, 1.0 %, 0.0 %) of the runs.

Phase 1 shows us that the $\pi_{\text{def},0}$ RL defender struggles against the trained RL attacker $\pi_{\text{att},1}$. The attacker hits the OI in (62.9 %, 52.8 %, 92.4 %) of the runs, whereas the defender manages to capture the attacker in only (32.5 %, 33.2 %, 6.1 %) of the cases. This is a stark difference from the results in phase 0.

Phase 2 shows us that the defender manages to relearn a policy $\pi_{\text{def},1}$ RL that successfully plans against $\pi_{\text{att},1}$ and makes up for the struggles from phase 1. Here the defender manages to capture the attacker in a staggering (95.4 %, 95.7 %, 97.5 %) of the cases whereas the attacker only hits the OI in (1.0 %, 1.1 %, 0.9 %) of the cases.

The results are consistent along the fully observable and partially observable regimes, with the results in the noisy partially observable regimes described in table ?? being the most stratified as they show the least amount of standoff cases. Overall, these results show that the defender not only learns how to become stronger and more versatile across different phases in the phased PPO training, but that this result holds within the partially observability pipeline via the EKF as long as one keeps sensing noise minimal.

Furthermore, front loading most of the compute for the policies during the training itself also results in much faster performance during simulation and rollout. For the results above on the last phase of training, we achieved a simulation time per sampling step of $(0.740 \pm 0.386, 0.999 \pm 0.285, 0.915 \pm 0.139)$ milliseconds during the rollout loop, which translates to $(0.952 \pm 0.473, 1.206 \pm 0.335, 1.081 \pm 0.198)$

milliseconds. To contrast these numbers, Liu et al.’s work on classical multi-agent orbital pursuit-evasion method claims to have achieved simulation compute times of between 159.427 and 168.725 ms per sampling step across their training scenarios. This showcases the computational efficiency advantages this method.

6.4 Analysis of results anecdotes of development

The results in section ?? show that the proposed method for adversarial reinforcement learning-based GNC is not only feasible but immensely successful in zero-sum strategy games with continuous control. We strongly believe that this finding is, in big part, due to the fact that the research problem presented in this work is properly constrained:

6.4.1 The well-defined, zero-sum objective definition via the reward function and the episodic termination conditions properly guide agents towards learning coherent policies

The objectives that both spacecraft have to fulfill are well defined, if not over-defined, by the reward function plus the episodic terminal conditions. The zero-sum framing of this problem helps the agents have these defined objectives.

In fact, early on in this research we focused on stand-off games instead of zero-sum games, where the defending agent would have to defend the asset from the attacker without docking onto the attacker, and both agents would be penalized if they docked/crashed into each other. This is inherently a non-zero sum game with shared equality constraints and an open loop, standoff objective.

The result was fairly underwhelming after months of research, where we couldn’t develop successful defender policies. Our conclusion was that you should try to solve not solve open-loop and undefined learning objectives with reinforcement learning. Instead, open-loop Nash equilibrium solvers and other classical optimal control frameworks might best suit these sorts of research tasks.

6.4.2 The constrained volume in space in which the agent is allowed to learn its policies allows for excellent learning transitions

The learning problem is properly constrained to a volume in space. We don't have the agents' policies search for solutions across infinite or undefined volumes in space. Instead, by defining arena boundaries, we're able to constrain the agent's learning transitions within a finite area in space.

By training within a confined volume in close-proximity pursuit evasion, the agents are forced to train from extremely dynamic behaviors. In the early development of this work, we tried to train policies exclusively using the larger arena (using a 100 m radius arena during training rather than a 20m radius one) and we found that this wouldn't work. Training from a 100m radius arena from the get go would be an extremely hard task for the agents, especially the defender, and would produce bad policies .

We suspect that part of this comes from the fact that training from long-proximity for the problem presented in this work may have to do with the long transient periods in these arena's where the agents are not really forced to interact with each other given much greater amounts of empty space in the arena. Part of this might be able to be somewhat solved by doing very fine tuning of the learning hyperparameters, be it from more environments, data, exploration, or some other means. This way agents could properly find solutions within the larger training environment that match one-to-one with those of the smaller training arena, but it probably entail much more time and resource intensive.

6.4.3 The trained policies to properly interpolate to larger arenas during evaluation, even if it was never trained on observations corresponding to larger arenas.

Expanding on the point above, we show that learning in a small, constrained volume as your environment gives identical results in larger volumes using the same learning policies. This is a rather fascinating find, as we show that the learned policies

don't overfit to the volume and then panic and don't know what to do when seeing observations outside of that volume. We believe that this is in big part thanks to the symmetry of the research problem proposed in this paper.

Instead of overfitting to the smaller arena's training data used during training, the trained policies properly interpolate agentic behavior learned in the smaller arena into the larger space, even though it's never seen the observations corresponding to the larger space during training.

By training in the smaller arena volume we manage to concentrate and condense all the agent's learning transitions into a relatively cheaper training that can then work in those larger arenas provided the symmetry.

6.4.4 The velocity saturation controller properly bounds the agent's behavior within a "safe" threshold without preventing learning

The CFB-QP velocity saturation controller properly safeguards and bounds the agents' actions to a defined limit. In fact, the learned policy clearly learns how to work within the bounds of the controller and somewhat coexist with it.

The development of this finding came from initial testing of learned policies without the controller on larger arenas. We'd notice that agent policies would act greedily pick up a lot of speed, and lose control. This meant that the defender would end up letting the attacker through a lot. After doing a lot of analysis on this phenomenon, we concluded that the reward function, as presented in section ??, does not really provide an incentive for the agent to safeguard its own actuation for the sake of keeping controllability within a certain threshold.

Alternatively, one could have put in these incentives into the reward functions, but we thought that it would have made for a more difficult learning objective as it would convolute with the terms currently present the reward function. This, in turn, would have made us lose interpretability of the reward function, meaning the ability to know what term is causing trouble if learning is good or bad, during policy training

or evaluation.

Instead, we show that following a modular framework like the one we suggest keeps the guidance learning objectives simple and the velocity controller separate. This not only, helps remove system-level ambiguities during development, but it proves our point that classical controllers can coexist with learned modules.

6.4.5 The policies trained in LTI dynamics translate to LTV dynamics with the provided velocity and acceleration bounds

The results trained under an HCW linear-time-invariant dynamics model hold within the context of an elliptical linear-time-variant dynamics model with almost equal performance.

We think that this is due to a couple of things: 1) the fact that the policies are trained on observations that don't depend on the dynamics model itself, and rather just the states of each agent and their adversary, and 2) the HCW or Elliptical LTV dynamics objectively don't provide that much environmental acceleration onto the agents relative to how much actuation limits we're giving each agent.

Ergo, the agents may be able to offset whatever environmental dynamics given the actuation limits that we're giving them in this work. If this work were to be translated to real life ultra low thrust ion thrusters, like those that can be found in CubeSats and small satellites of the sort in LEO, this is one of the things I'd be most careful with.

6.4.6 Trained policies hold under partial observability and sensor noise when trained via behavior cloning from the full states

The behavior cloning piece shows that all of the conclusions above holds up under partial observability. By teaching student policies based on the observations measured by the bearings-only EKF, we have minimal performance loss from the fully observable results.

Of course, to point it out once more, this is assuming that you can somehow measure the prior action of your adversary within a certain amount of noise through some other means. Or if using some other measurement strategy not dependent on bearings-only EKF, for your adversarial state measurements to be highly accurate.

Regardless, it is exciting to find that we can develop successful policies for noisy, partially observable adversarial RL with

6.5 Analysis of ideas for future development

For follow ups on the work presented on this thesis, however, I think that there are a number of areas that could be researched to expand on the scopes of what is presented here. Some ideas for this include.

- 1) Modifying the reward functions to include rewards for causing poor measurements on opponents (i.e: By rewarding for the higher the ??? in the ??? matrix of the opponent would be per certain dynamics and the sort).
- 2) Allowing the asset to be able to move along the dynamics model instead of having it be static at the chief of the orbit.
- 3) Adding more elaborate, realistic control models to simulate the actuators on board the spacecraft with high fidelity.

It must be noted that the work presented in this thesis is meant to be an extensive proof of concept of certain ideas that could make RL-based GNC interesting in theory and simulation. The zero-sum game presented in this work is just meant to allow us to be a showcase how these RL can be used, whereas there are many other potential uses for these technologies.

The whole point of this thesis is how one can use RL derive deterministic policies for stochastic and unpredictable behaviors within an aerospace context. Even

in planning without adversarial opponents, there's other areas where RL-based planning would prove to be resourceful, such as:

- 1) To assist in Earth or celestial-bound terrain relative navigation of aircraft and spacecraft across well-defined start and end points on uncertain terrain.
- 2) To aid in planning on precision spacecraft landings in GPS-denied environments upon loss of redundancy or function of core navigation sensors. (e.g: laser-based altimetry)
- 3) Adding more elaborate, realistic control models to simulate the actuators on board the spacecraft with high fidelity.

All in all, as long as the principles in section ??? are respected, regarding the concrete and realistic objectives that shall be given to RL agents for them to learn appropriate policies, the potential applications for RL to aid or replace classical GNC architectures are endless.

Chapter 7: Conclusion

Throughout this thesis we showed that the proposed (NAME PENDING) method RL-based adversarial planning can work within the constraints of a navigation and control architecture. With the appropriate hyperparameters, the method delivered convergent policies throughout via phased learning with opponent randomization across a number of testing scenarios.

Our Monte Carlo evaluations of the resulting policies indicate that, in a Target-Attacker-Defender constrained zero sum game with 1 defender vs 1 attacker, we achieved highly performing defender policies that succeed between ???% and ???% in both fully observable scenarios and partially observable scenarios, the latter which the agents are forced to take bearings-only measurements to obtain their opponents' states. This was done across 4800-9600 Monte Carlo runs per arena. The compute time for these pre-trained policies during inference and rollout was were ??? ms, providing fast compute speeds that showcase the potential computational advantages of such a method not only in adversarial space scenario but in other space autonomy regimes where fast compute times are required.

These results helped us confirm our initial hypothesis to conclude that RL architectures can be used in zero-sum adversarial scenarios, can learn coherent policies while subject to the constraints of classical navigation filtering and control in the loop, can remain robust under full and partial observability, interpolate outside of the training dataset given a symmetric problem, and provide convergent and highly performing behavior despite the stochasticity.

Overall, this project showcases that, as long as they're applied within contexts with well-defined objectives, pure deep reinforcement learning based methods have a feasible implementation in certain areas of aerospace decision making, without needing to resort or fallback to classical optimization methods in the process. The fact

that we showed that these results hold under partial observability justify a potential hardware application for algorithms along this line of work beyond the theoretical experiment. This could open the door for further implementations of such aerospace decision-making algorithms in the future as well as their coupling with existing classical control architectures.

Appendix A: Monte Carlo Dispersions

Text here.

A.1 Appendix section

Text here.

A.1.1 Appendix section

Text here.

A.1.1.1 Appendix section

Text here.

Appendix B: Another Appendix

Glossary

Gus Santaella A hard-working graduate student, nearing the end of this chapter of your education!

Master of Science in Engineering Gus Santaella is about to earn this. Congratulations!

Works Cited

Vincent J. Aidala. Kalman filter behavior in bearings-only tracking applications. *IEEE Transactions on Aerospace and Electronic Systems*, AES-15(1):29–39, 1979. doi: 10.1109/TAES.1979.308793.

Aaron D. Ames, Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017. doi: 10.1109/TAC.2016.2638961.

X. Cao, X. Ning, X. Lian, T. Zhang, S. Li, G. Zhang, and Z. Chen. A non-fixed-duration turn-based orbital game algorithm based on history-aware and predictive result reward proximal policy optimization. *IEEE Transactions on Aerospace and Electronic Systems*, 2026. doi: 10.1109/TAES.2026.3657323. Accepted / in press.

W. H. Clohessy and R. S. Wiltshire. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences*, 27(9):653–658, 1960. doi: 10.2514/8.8704.

H. Gong, S. Gong, and J. Li. Pursuit-evasion game for satellites based on continuous thrust reachable domain. *IEEE Transactions on Aerospace and Electronic Systems*, 56(6):4626–4637, 2020. doi: 10.1109/TAES.2020.2998197.

George William Hill. Researches in the lunar theory. *American Journal of Mathematics*, 1(1):5–26, 1878.

Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME—Journal of Basic Engineering*, 82(Series D):35–45, 1960.

X. Li and X. Wang. Online solution for orbital pursuit-evasion game via heterogeneous proximal policy optimization. *IEEE Transactions on Aerospace and Electronic Systems*, 61(5):12044–12058, 2025. doi: 10.1109/TAES.2025.3570663.

Y. Liu, Y. Zhang, J. Jiang, and C. Li. Multiple-to-one orbital pursuit: A computational game strategy. *IEEE Transactions on Aerospace and Electronic Systems*, 61(2):2213–2225, 2025. doi: 10.1109/TAES.2024.3471497.

Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635. PMLR, 2011.

Max Rudolph, Nathan Lichtle, Sobhan Mohammadpour, Alexandre Bayen, J. Zico Kolter, Amy Zhang, Gabriele Farina, Eugene Vinitsky, and Samuel Sokota. Reevaluating policy gradient methods for imperfect-information games. *arXiv preprint arXiv:2502.08938*, 2025. doi: 10.48550/arXiv.2502.08938.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347.

J. Selvakumar and E. Bakolas. A pursuit-evasion game in the orbital plane. In *27th AAS/AIAA Space Flight Mechanics Meeting*, 2017. AAS 17-474.

J. Tschauner and P. Hempel. Rendezvous zu einem in elliptischer bahn umlaufenden ziel. *Astronautica Acta*, 11(2):104–109, 1965.

J. Zhang, K. Zhang, Y. Zhang, H. Shi, L. Tang, and M. Li. Near-optimal interception strategy for orbital pursuit-evasion using deep reinforcement learning. *Acta Astronautica*, 198:9–25, 2022. doi: 10.1016/j.actaastro.2022.05.057.

Vita

Gus Santaella is going to graduate soon!

Address: gussantaella@utexas.edu

This thesis was typeset with $\text{\LaTeX}^\dagger$ by the author.

[†] $\text{\LaTeX}$ is a document preparation system developed by Leslie Lamport as a special version of Donald Knuth's $\text{\TeX}$ Program.